1. Introduction

1.1 College Profile

1. Bhagwan Mahavir College of Computer Application, Surat a highly commendable private institute, occupies a place amongst the technical institutes of the southern region of Gujarat. It is established in year 2008, financed and run by the Bhagwan Mahavir College Foundation, Surat. It is affiliated to Bhagwan Mahavir University. The college is controlled by the executive board formed from the representatives of the Trust under constitution & by laws of the college. The college campus is located at the open green field areas with a beautiful location, situated at New city light Road, Bharthana – Vesu , Surat.

2. The College has excelled itself in many fields. It has occupied a prominent place in the education sphere of South Gujarat. The building surrounded by green fields and contains spacious classrooms, well documented library, a multipurpose hall, modern computer lab. It has at present around 850 students.

a. Vision

- Bhagwan Mahavir College of Computer Application aims to become diversified, market leaders and preferred institution within the country as well as global repute for the holistic development of community in the domain of Information Technology/Computer Science.

b. Mission

Major focus on holistic development of student's career:

- To become the diversified and multidisciplinary institution in the domain of Computer Application/Science.
- To Develop the life-long community.

- To keep curriculum, Teaching & Learning pedagogy dynamic, agile, innovative and industry responsive.
- To focus more on Outcome based learning, industry collaboration and encouragement for entrepreneurship, innovation and start-ups.

c. Objectives

- The College of Computer Application shares University's overall mission of being a centre of academic excellence by providing qualitative education and integrating training. The progressive and innovative faculty, through their teaching, scholarship and publication, leadership and service are fully committed to the provision of an educational environment which prepares individuals to become successful professionals.
- The main objective of the Department is to help students meet their long-range needs as maturing professionals as well as their immediate needs upon entering the profession.

1.2 Project Profile

PROJECT TITLE	BHARATSTOCK: RETAIL MANAGEMENT SYSTEM
PROJECT DESCRIPTION	This project aims to develop a sophisticated, cross-platform retail management solution using the Flutter framework and Dart programming language. The app provides the shop owners a paper free solutions to there shops to make there work easy. It covers inventory management, billing, customer & supplier ledger, staff attendance & payroll, expense tracking, and multi-language support (english, hindi, gujarati).
DURATION	2 MONTHS
FRONT END	FLUTTER
BACK END	FIREBASE
LANGUAGE	DART
PROJECT GUIDE	ASST PROF. JAHNVI B MASRANI
SUBMITTED BY	Mr. RAHUL JAGLAN Mr. PATIL HIMANSHU GHANSHAM Mr. BACHHAV MANISH DIPAK Mr. ITALIYA MANAV PRAVINBHAI

2. Proposed System

2.1 Scope & Objective

The Proposed System, titled **BHARATSTOCK: Retail Management System**, is designed to be a comprehensive digital ecosystem that redefines how small-scale retail and wholesale enterprises manage their day-to-day operations. The core objective is to bridge the gap between traditional " book-Khata" (manual bookkeeping) and modern Enterprise Resource Planning (ERP) systems.

1. **Scope**:

BharatStock is designed to be a comprehensive, 360-degree digital solution for Indian retail operations, focusing specifically on the needs of small and medium-sized business owners who face daily challenges in managing stock, computing taxes, and maintaining financial records. The scope of the system is defined by five major operational pillars, each addressing a critical gap in the current landscape of retail management in India:

Authentication & Multi-Tenancy:

Every business registered on BharatStock is isolated within a secure businessId sandbox maintained within Google Cloud Firestore. This architecture ensures that even when thousands of businesses simultaneously use the platform, each business's sensitive financial data, stock records, and client information remain completely private, protected, and inaccessible to other users. Multi-tenancy is achieved at the database-rules level using Firebase Security Rules.

Inventory & Stock Auditing:

BharatStock includes a real-time inventory management module that tracks stock levels, purchase prices, selling prices, and profit margins for every product. The system includes automated low-stock alert notifications to prevent stock-out situations, which are a common cause of lost revenue for small retailers. Product data can be imported in bulk or added individually with HSN codes for tax compliance.

Billing & Invoice Dispatch:

The application features a powerful digital billing engine capable of generating high-quality PDF invoices with complete GST breakdowns (CGST, SGST, IGST). These invoices are automatically pushed to a B2B Interaction Hub, where supplier counterparts can view and approve incoming bills, enabling a streamlined digital supply chain workflow.

Analytics & Financial Intelligence:

A dedicated analytics module tracks monthly profit/loss summaries, GST liability calculations, Input Tax Credit (ITC) reconciliation, and best-selling product identification — providing owners with actionable business intelligence without requiring any knowledge of accounting.

2. **Objectives:**

The primary objectives of BharatStock are carefully crafted to address the real-world pain points experienced by Indian retail business owners on a daily basis:

a. Financial Digitization: To completely eliminate the losses associated with paper-based record-keeping, including records damaged by fire, water, or physical theft. BharatStock stores all financial data securely on Google's encrypted cloud infrastructure, ensuring permanent, accessible, and tamper-proof records.

b. Accuracy in Tax Compliance: To automate the highly complex and error-prone process of computing GST splits between intra-state transactions (CGST + SGST) and inter-state transactions (IGST), ensuring that the retailer never underpays or overpays GST amounts. The system auto-detects the appropriate tax category based on the buyer's and seller's state codes.

c. Universal Accessibility: By supporting multiple regional languages including Gujarati and Hindi in addition to English, BharatStock is designed to reach business owners in rural, suburban, and semi-urban areas of India who are not comfortable with English-only interfaces.

d. GSTR Compliance Reporting: The system auto-generates GSTR-1 and GSTR-3B compatible reports, dramatically reducing the time and cost associated with quarterly and monthly GST filing.

e. Improved Cash Flow Management: Through real-time ledger tracking, owners can at any moment see exactly which customers owe them money (outstanding receivables) and which suppliers they need to pay (outstanding payables), enabling proactive cash flow management.

2.2 Advantages

1. **Mobility and Remote Access**

One of the most significant advantages of BharatStock is its mobile-first design. Business owners are no longer tethered to a specific physical location or device to access their shop's operational data. Whether traveling for procurement, attending a trade fair, or simply away from the shop, owners can monitor real-time stock levels, approve invoices, review daily sales reports from anywhere in the world using their smartphone.

2. **Tactile User Interface with Claymorphism**

BharatStock's interface is built on the Claymorphic Design Pattern — a modern evolution of UI design that creates visually 'inflated', soft, and three-dimensional UI components using layered shadows, rounded corners, and pastel color palettes. Unlike flat design interfaces, which can be difficult to parse for users with visual impairments or limited digital literacy, Claymorphism makes interactive elements immediately visually distinct, reducing learning curves and minimizing user errors. This is critically important for reaching the large segment of Indian shop owners who may have limited prior experience with digital tools.

3. **Automated GSTR Reports**

Prior to BharatStock, compiling a GST return for a small retailer typically involved hours of manual data entry, error checking, and cross-referencing with physical bills — a process that often occupied an entire weekend and sometimes required hiring a dedicated accountant. BharatStock reduces this to a single button click. The

system maintains a continuous, structured log of every sale, its GST category, tax component, and party information, which is then formatted into standard GSTR-compatible reports with zero additional input from the user.

4. **Scalability Across Business Sizes**

   BharatStock is architected to scale seamlessly from a single-counter street-corner shop to a multi-staff warehouse operation with hundreds of product lines. Because the system is built on Firebase's NoSQL document database and Google Cloud Functions, there is no upper limit on the volume of transactions, users, or products the system can handle. Business owners who start small and grow rapidly do not need to migrate to a different platform — BharatStock grows with them.

2.3 Feasibility Study

Prior to commencing development, a thorough feasibility analysis was conducted across three primary dimensions — technical, economic, and operational — to validate that BharatStock could be successfully built, deployed, and sustained within the project's constraints and timeline.

2.3.1 Technical Feasibility

The development of BharatStock is built upon the Flutter + Firebase technology stack, which is one of the most well-documented, widely-supported, and actively maintained cross-platform development ecosystems available today. The technical feasibility of this project is extremely high for the following reasons:

1. Hot Reload Development Efficiency: Flutter's signature Hot Reload feature allows developers to instantly see the result of code changes in the running application without restarting it. This dramatically accelerates UI iteration, especially during the complex task of building and fine-tuning Claymorphic components.
2. NoSQL Flexibility for Evolving Tax Rules: Indian GST regulations are subject to periodic amendment by the GST Council, which may introduce new tax categories, rates, or compliance requirements (such as Reverse Charge Mechanism, e-Way Bills, or HSN code changes) with relatively short notice. Cloud Firestore's schema-less NoSQL structure allows the database to accommodate these changes without requiring a complete system redesign or data migration.
3. Serverless Logic with Cloud Functions: Firebase Cloud Functions enable the execution of server-side business logic — such as generating unique, non-repeating Invoice IDs, computing ITC eligibility, and sending push notifications — without requiring BharatStock to maintain, patch, or pay for dedicated server infrastructure.
4. Cross-Platform Reach: A single Flutter codebase compiles natively to both Android and iOS applications, allowing BharatStock to reach the full spectrum of Indian smartphone users regardless of their device ecosystem.

2.3.2 Economical Feasibility

From a financial perspective, BharatStock represents a highly cost-effective solution to develop, deploy, and scale:

1. Zero Server Infrastructure Cost: The entire application backend runs on Google's distributed cloud infrastructure through Firebase. There are no physical or virtual servers to purchase, configure, maintain, or secure. Firebase's pricing model scales proportionally with actual usage, meaning early-stage deployments with low transaction volumes incur minimal costs.
2. Firebase Free Tier: Firebase's Spark (free) plan provides generous quotas for Firestore reads/writes, Cloud Functions invocations, and Firebase Authentication, which are more than sufficient for a pilot deployment or small-to-medium business deployment.
3. Reduced Accountant Dependency: By automating GST computation, report generation, and ledger maintenance, BharatStock can reduce or eliminate a small retailer's dependency on a hired accountant for routine compliance tasks, potentially saving tens of thousands of rupees annually.
4. Flutter's Single-Codebase Advantage: Maintaining a single Dart codebase for both Android and iOS eliminates the cost of employing or contracting separate native development teams for each platform — a significant financial advantage for a project of this scale.
5. Open Source Ecosystem: All core frameworks and tools used in BharatStock — Flutter, Dart, Firebase SDKs, the pdf package, the intl package — are available under open-source licenses at no cost, dramatically reducing software licensing overhead.

2.3.3 Operational Feasibility

Operationally, BharatStock is designed from the ground up to integrate seamlessly into the existing workflows, digital habits, and communication patterns of Indian retail business owners:

1. Simple Onboarding Process: New staff members can be added and made operational within minutes by simply entering their phone number. The role-based permission system ensures they immediately see only the screens and functions relevant to their job responsibilities.
2. Language Support: Real-time language switching between English, Hindi, and Gujarati means that even staff members with limited English proficiency can operate the system effectively.
3. Minimal Hardware Requirements: BharatStock requires only a standard Android or iOS smartphone — devices that virtually every Indian retail shop owner already possesses — and a basic internet connection. No specialized hardware investments are required.

3. System Analysis

System analysis is the systematic investigation and decomposition of an existing problem domain and the modeling of a proposed solution. It is arguably the most critical phase of the entire software development lifecycle, as errors or omissions made during analysis propagate forward into design and implementation, where they become exponentially more expensive to correct.

For BharatStock, the system analysis phase involved direct observation of retail billing workflows, extensive interviews with Kirana shop owners in Surat's commercial areas, competitive benchmarking against existing solutions such as Vyapar, MyBillBook, and TallyPrime, and review of the Government of India's GST digitalization mandate documents.

3.1 Existing System (Manual)

The term 'Khata' (meaning 'ledger' or 'account book') describes the traditional paper-based bookkeeping system that has dominated Indian retail for centuries and continues to dominate the daily operations of millions of small shop owners. Despite the significant advances in digital technology, a vast majority of Kirana stores, hardware shops, and wholesale traders continue to rely exclusively on handwritten registers for their critical business records. An analysis of this existing system reveals a comprehensive set of systemic failures that directly motivated the development of BharatStock:

1. Physical Vulnerability of Records: Paper registers are inherently fragile. A single incident — fire, water damage from monsoon flooding, theft, or simple accidental destruction — can permanently erase years of irreplaceable financial records. There is no backup, no version history, and no recovery mechanism. Many shop owners interviewed during the research phase recounted instances of significant financial loss directly attributable to destroyed paper records.
2. Manual GST Calculation Errors: The Indian GST system requires the correct application of five different tax slabs (0%, 5%, 12%, 18%, and 28%) across thousands of product categories, each with a specific HSN code. When calculating manually, errors in applying the correct rate or in computing CGST vs. SGST splits are extremely common. Such errors can result in GST penalties, notices from tax authorities, and loss of Input Tax Credit eligibility.
3. Absence of Real-Time Inventory Visibility: With a paper system, a shopkeeper has no real-time mechanism to know the precise current stock level of any product. Stock counts are performed periodically — often weekly or monthly — meaning that stock-outs are frequently discovered only at the point of a customer asking for an unavailable item. This leads to lost sales and customer dissatisfaction.
4. Inability to Search Historical Records: Retrieving a specific invoice from six months prior in a stack of physical register books can take anywhere from thirty minutes to several hours. This makes dispute resolution with customers, responses to tax authority queries, and credit verification of trade partners extremely time-consuming and operationally disruptive.

3.2 Need for New System

The inadequacies of the existing Khata system, combined with several powerful external forces, create a compelling and urgent need for a modern digital replacement:

1. Regulatory Mandate: The Government of India's GST digitalization initiative mandates electronic invoice generation and submission for businesses above certain turnover thresholds. As these thresholds are progressively lowered, an increasing proportion of small retailers will soon be legally required to maintain digital invoicing systems.
2. Customer Expectations: Customers in urban and semi-urban India increasingly expect digital receipts, the ability to track purchases, and the convenience of WhatsApp-based billing. Business owners who cannot provide these experiences risk losing customers to more digitally capable competitors.
3. Financial Insights for Survival: In a highly competitive retail environment with thin margins, the ability to quickly identify which products are profitable, which are underperforming, and where costs can be optimized can mean the difference between business survival and closure. BharatStock puts this intelligence directly in the owner's hands.
4. Post-Pandemic Digital Shift: The COVID-19 pandemic accelerated digital adoption across all segments of Indian business. Many shop owners who previously resisted digital tools now recognize their value and are actively seeking solutions that are simple, affordable, and tailored to Indian business practices.

3.3 Detailed SRS (Software Requirement Specification)

1. Introduction

This Software Requirements Specification (SRS) document provides a complete and unambiguous description of all functional and non-functional requirements for the BharatStock Retail Management System. It is intended to serve as the authoritative reference document for all development, testing, and validation activities associated with this project. The SRS is written in accordance with the IEEE 830 standard for software requirements documentation.

2. Functional Requirements

Req. ID	Requirement Name	Description
FR.1	User Authentication	Secure login via email/password with OTP verification. Support for role-based login directing users to role-appropriate screens.
FR.2	Shop Onboarding	Owner must be able to register their full business profile: GSTIN, trade name, registered address, state code, and business logo.
FR.3	Inventory CRUD	Full Create, Read, Update, Delete operations for products including HSN code, GST rate, purchase price, sell price, and current stock quantity.

Req. ID	Requirement Name	Description
FR.4	Billing Engine	Ability to add products to a cart, apply percentage or flat discounts, auto-compute CGST/SGST/IGST based on buyer's state, and generate PDF invoice.
FR.5	Party Management	Manage customers and suppliers (parties) with contact details, GST info, billing/shipping addresses, and real-time balance tracking. Record payment transactions against each party. Support for both registered and unregistered GST entities.
FR.6	Analytics Dashboard	Monthly profit/loss trends, GST liability summary, ITC tracking, best-selling products chart, and outstanding receivables/payables.
FR.7	Localization	On-the-fly language switching between English, Hindi, and Gujarati without app restart.
FR.8	Staff, Attendance & Payroll	Owner can add staff with salary details. Mark daily attendance (Present/Absent/Half-Day). System auto-calculates monthly salary based on attendance ratio and allows marking salary as paid.

3. Non-Functional Requirements

Req. ID	Category	Requirement
NFR.1	Security	All business data must be isolated using Firebase Security Rules. No cross-business data access permitted under any authenticated session.
NFR.2	Performance	Real-time Firestore listeners must reflect database changes in the UI within 150 milliseconds under normal network conditions.
NFR.3	Usability	The UI must maintain a minimum of 60 frames per second during all scroll, animation, and transition interactions on mid-range Android devices.
NFR.4	Maintainability	Codebase must follow SOLID principles and be organized into feature-based modules to allow independent development and testing of each module.
NFR.5	Scalability	The system architecture must support expansion to 10,000+ concurrent business users without architectural changes.

4. System Planning

Before undertaking any software development effort, it is essential to establish a clear, structured plan that defines what needs to be built, who will build each component, and in what sequence the work will be completed. Effective project planning serves as the backbone of successful software delivery, preventing common failure modes such as scope creep, resource underutilization, and missed deadlines.

4.1 Requirement Analysis & Data Gathering

1. **Methodology**

   The requirement analysis for BharatStock employed a multi-method approach designed to capture a comprehensive and representative picture of the real-world retail management challenges faced by the target user base:

   a. User Interviews: In-depth, semi-structured interviews were conducted with owners of Kirana stores and textile traders operating in Surat districts. These interviews focused on understanding daily billing workflows, common pain points with paper-based systems, attitude toward digital adoption, and feature priorities.

   b. Contextual Inquiry: Researchers spent time directly observing billing operations at retail counters, watching how bills are manually calculated, how GST is applied, how customer queries are handled, and how daily closing of accounts is performed. This observational approach revealed several implicit requirements that the shop owners themselves had not articulated.

   c. Regulatory Research: The GST Council's published guidelines, GSTR filing formats, and e-invoicing mandate documentation were studied to ensure that BharatStock's billing engine and reporting module would be compliant with both current and anticipated future regulatory requirements.

2. **Data Sources & Validation**

   Requirement data was collected from a sample of 2 shop owners representing a diverse cross-section of retail categories (grocery, hardware, textile, electronics), business sizes (annual turnover ranging around ₹3 lakh), and digital literacy levels (complete novice to basic smartphone user). Requirements were validated through two rounds of prototype feedback sessions, during which early wireframe designs were presented to a subset of the interviewed shop owners for critique and refinement.

3. **System Constraints & Considerations**

   The following constraints were identified and factored into the system design during the planning phase:

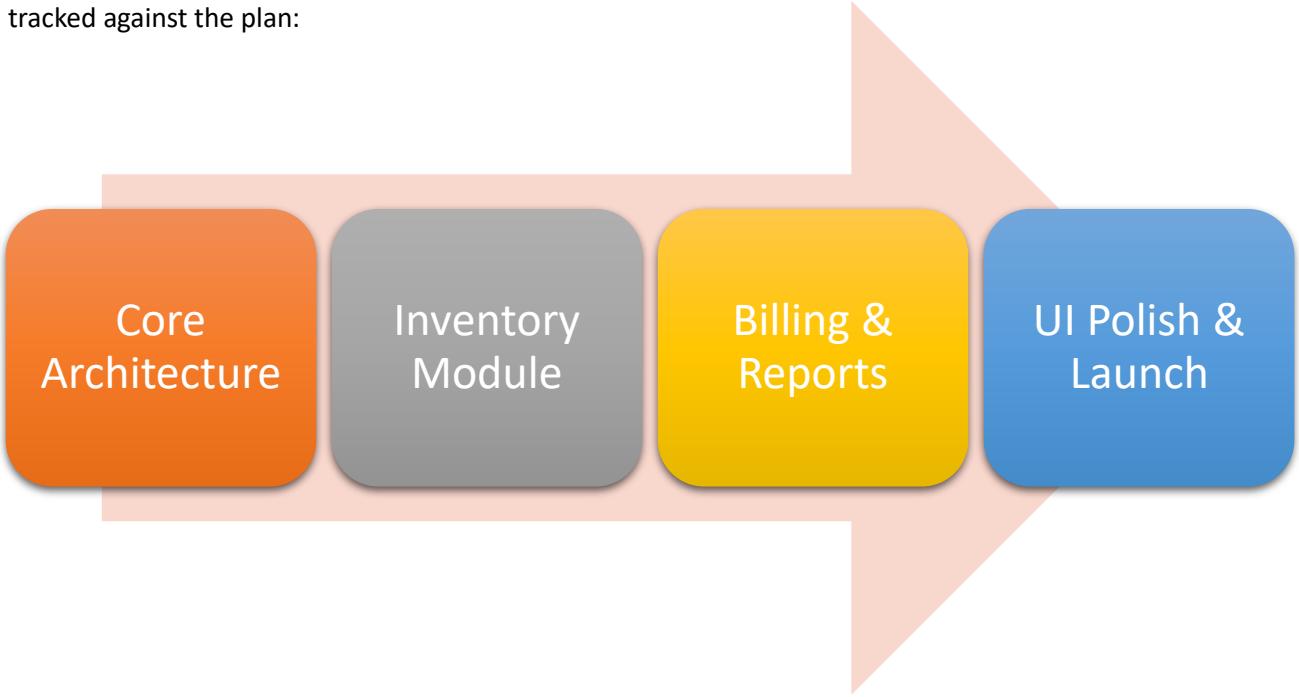
   a. Connectivity Dependency: Real-time synchronization of billing and inventory data requires internet access. A minimum 4G mobile data connection is needed to save and sync transactions. Offline queuing is provided for resilience.

   b. Platform Priority: The initial release targets Android, which commands approximately 95% of the Indian smartphone market. iOS support is planned as a secondary release phase.

   c. Firebase Free-Tier Limits: For the pilot deployment, the application is designed to operate within Firebase's Spark (free) tier limits, necessitating careful optimization of Firestore read/write operations.

4.2 Time-line Chart

The project followed a structured development cycle organized into four major phases, with weekly milestones tracked against the plan:



Phase	Month	Key Activities	Milestone
Phase 1: Core Architecture	Month 1 (Weeks 1–2)	Firebase project setup, Firestore schema design, Authentication module, Role-based navigation structure, Multi-tenancy implementation	Core data architecture and auth flow complete and tested
Phase 2: Inventory Module	Month 2 (Weeks 3–6)	Product CRUD screens, HSN code lookup, Stock level tracking, Low-stock alert logic, Inventory search and filter UI	Full inventory management module operational
Phase 3: Billing & Reports	Month 3 (Weeks 7–11)	Cart logic and discount engine, GST auto-computation (IGST/CGST/SGST), PDF invoice generation, WhatsApp sharing, GSTR report export, Analytics dashboard	Billing engine and report exports complete
Phase 4: UI Polish & Launch	Month 4 (Weeks 12)	Claymorphic UI refinement, Hindi/Gujarati localization, B2B Interaction Hub, Full system testing, Documentation, Final presentation preparation	Production-ready release with documentation

5. Tools & Environment Used

5.1 Hardware and Software Requirement

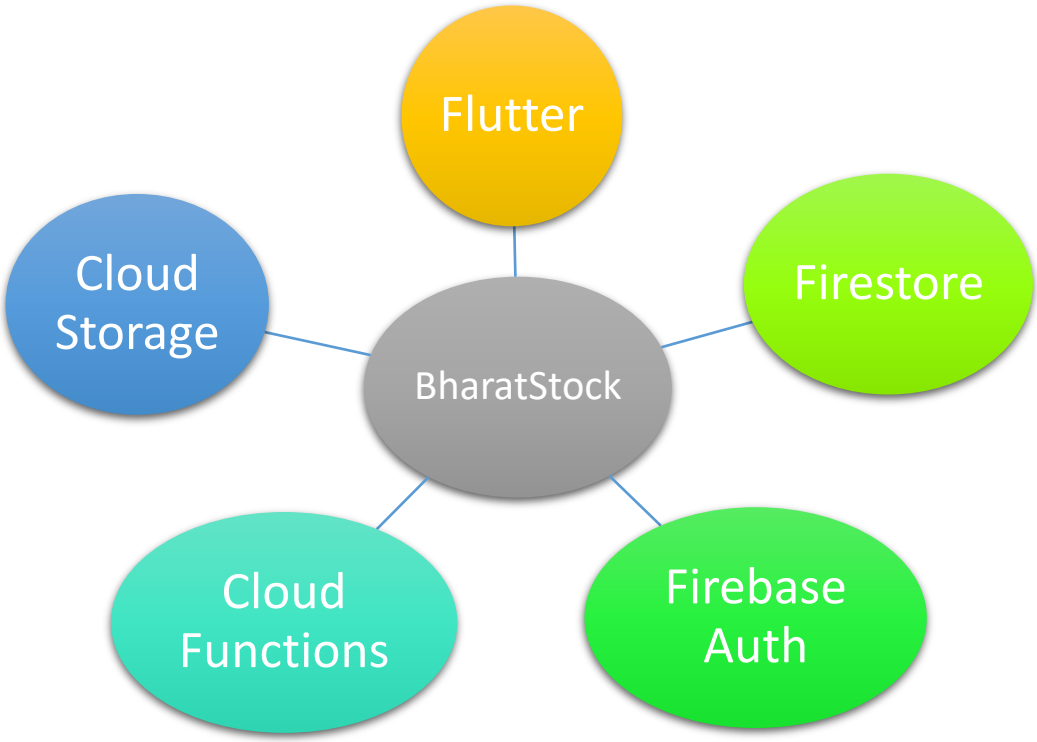
5.1.1 Software Requirement

Component	Specification
Operating System	Microsoft Windows 11
Primary Language	Dart (Version 3.x, stable channel)
Frontend Framework	Flutter SDK (Stable Channel, 3.x)
Backend Platform	Google Firebase (Firestore, Cloud Functions, Auth, Storage)
IDE	Android Studio / VS Code(ANTIGRAVITY) with Flutter & Dart plugins
Version Control	Git with GitHub
Documentation Tools	Microsoft Word

5.1.2 Hardware Requirement

Component	Minimum Specification
Processor	Intel Core i5 (10th Generation or higher) / AMD Ryzen 5 equivalent
Memory (RAM)	8 GB DDR4 (16 GB recommended for emulator performance)
Storage	256 GB SSD (Solid State Drive for fast build times)
Display	Full HD 1920×1080 for accurate layout design and preview
Operating System	Microsoft Windows 11 64-bit / macOS 12 (Monterey) or later
Test Device	Android smartphone running Android 8.0 (API 26) or higher

5.1.3 Technology to be used



1. Flutter SDK: Used for building the cross-platform Android and iOS application from a single Dart codebase.
2. Google Firestore: NoSQL cloud database for real-time data storage and synchronization.
3. Firebase Authentication: Secure, scalable user identity management with support for email/password and phone-based OTP login.
4. Firebase Cloud Functions: Serverless Node.js functions for secure server-side logic.
5. Firebase Cloud Storage: Storage for business data and generated PDF invoice files.

5.2 Server-Side and Client-side Tools

The Flutter Framework (Client-Side)

Flutter is an open-source UI software development kit created and maintained by Google, released in 2018. It enables developers to build natively compiled applications for mobile (Android and iOS), web, desktop, and embedded platforms from a single codebase written in the Dart programming language. Flutter's core architectural innovation is its custom rendering engine (based on Skia/Impeller) that draws every pixel of the UI directly to the canvas, completely bypassing the native platform's UI framework. This approach delivers two critical advantages for BharatStock:

1. Performance: Because Flutter renders entirely in its own engine rather than delegating to platform UI components, it achieves consistently high frame rates (60fps and above) regardless of the complexity of the

UI. This is essential for BharatStock's Claymorphic widgets, which involve layered shadow calculations and micro-animations.

2. Pixel-Perfect Consistency: The exact same visual output is produced on every device and every platform, eliminating the device-specific rendering inconsistencies that plague apps built with platform-native UI frameworks.

Flutter's widget-based composition model allows the BharatStock development team to build complex, reusable UI components — such as the GST-aware cart item card or the real-time stock level indicator — and compose them into full screens with minimal boilerplate code. The Dart programming language's strong typing, null safety system, and async/await concurrency model further contribute to the reliability and maintainability of the codebase.

Firebase (Server-Side)

Firebase is Google's comprehensive Backend as a Service (BaaS) platform, providing a unified suite of cloud infrastructure services designed to accelerate mobile and web application development. Firebase was originally founded as an independent startup in 2011 and was acquired by Google in 2014, after which it was expanded into the robust, enterprise-grade platform it is today.

For BharatStock, Firebase serves as the complete backend infrastructure, replacing the need for a traditional server-side application stack. The specific Firebase services utilized are:

1. Cloud Firestore: A highly scalable, serverless, NoSQL document database that provides real-time data synchronization across all connected clients. Firestore stores data in a hierarchical collection → document → subcollection structure, making it naturally suited to BharatStock's business → products/bills/staff hierarchy. Firestore's powerful Security Rules system enforces data access control at the database level, ensuring complete multi-tenant data isolation.
2. Firebase Authentication: Manages the full lifecycle of user identity, including registration, login, session token management, and secure password reset flows. For BharatStock, Firebase Auth handles the authentication of both Owners and Staff, with custom claims used to store role information in the JWT token that each user session carries.
3. Cloud Functions for Firebase: A serverless compute environment (running Node.js) that allows BharatStock to execute trusted server-side logic in response to Firestore database events, HTTP requests, or scheduled triggers. Key use cases include generating globally unique, sequential invoice IDs, sending automated push notifications when stock levels fall below minimum thresholds, and computing aggregated monthly financial summaries.
4. Firebase Cloud Storage: Object storage service used to persist business logo images and finalized PDF invoice files, which are then served to users via secure, expiring download URLs.
5. Firebase Cloud Messaging (FCM): A cross-platform push notification delivery service used to deliver real-time low-stock alerts, B2B interaction updates, and payment reminders to the app.

Dart Programming Language

Dart is the programming language used to write Flutter applications. Created by Google in 2011, Dart is a modern, strongly-typed, object-oriented language that compiles to native machine code for mobile and desktop platforms and to JavaScript for web deployment. For BharatStock, Dart's key features include its null safety type system, which eliminates an entire class of null-pointer runtime errors; its async/await and Future/Stream model, which simplifies the management of asynchronous Firestore operations; and its class-based inheritance model, which enables clean object-oriented domain modeling of business entities such as Product, Bill, and BillItem.

6. System Design

System design is the creative, architectural process of transforming the logical requirements defined in the SRS into a concrete, implementable blueprint for the software system. It encompasses the high-level structural design of the system's components and their interactions (architectural design), the detailed internal design of individual modules (module-level design), the structure of the data that the system will store and manipulate (database design), and the visual organization of the interfaces through which users interact with the system (UI design).

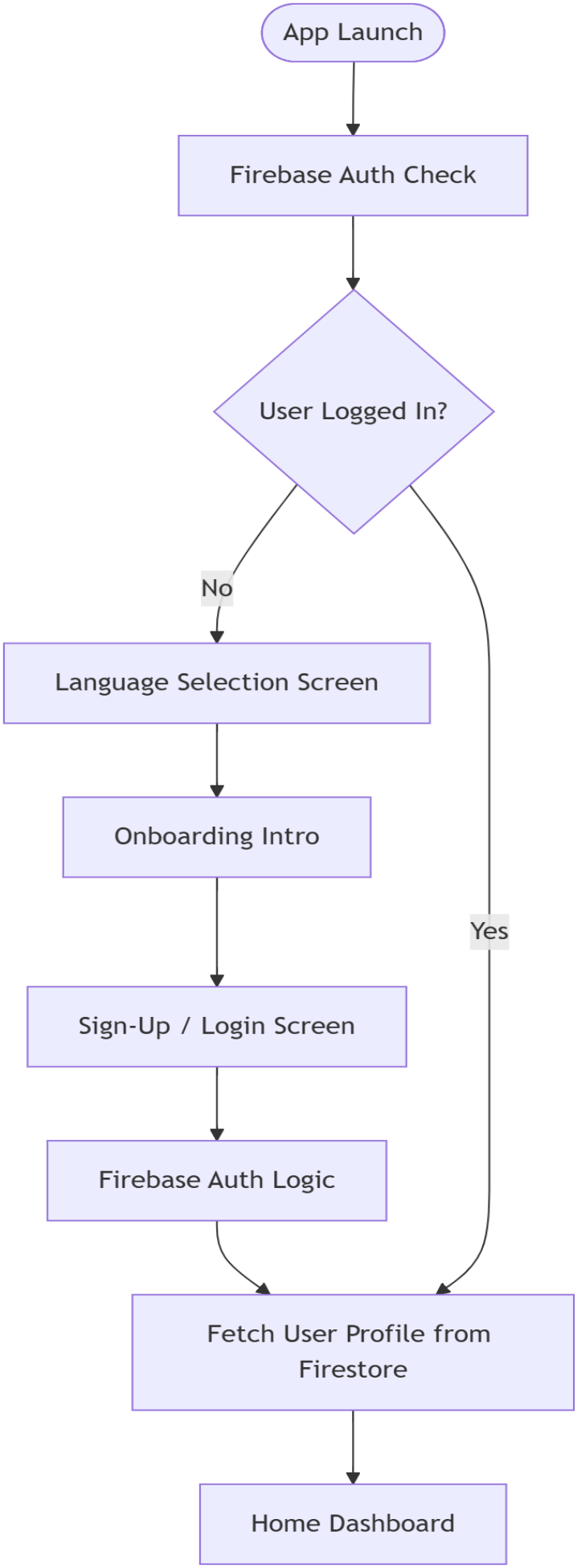
6.1 Unified Modeling Language (UML)

UML (Unified Modeling Language) is a standardized visual language used to represent the architecture, design, and behavior of a software system. BharatStock uses four core UML diagrams: Activity, Use Case, Class, and Sequence Diagrams.

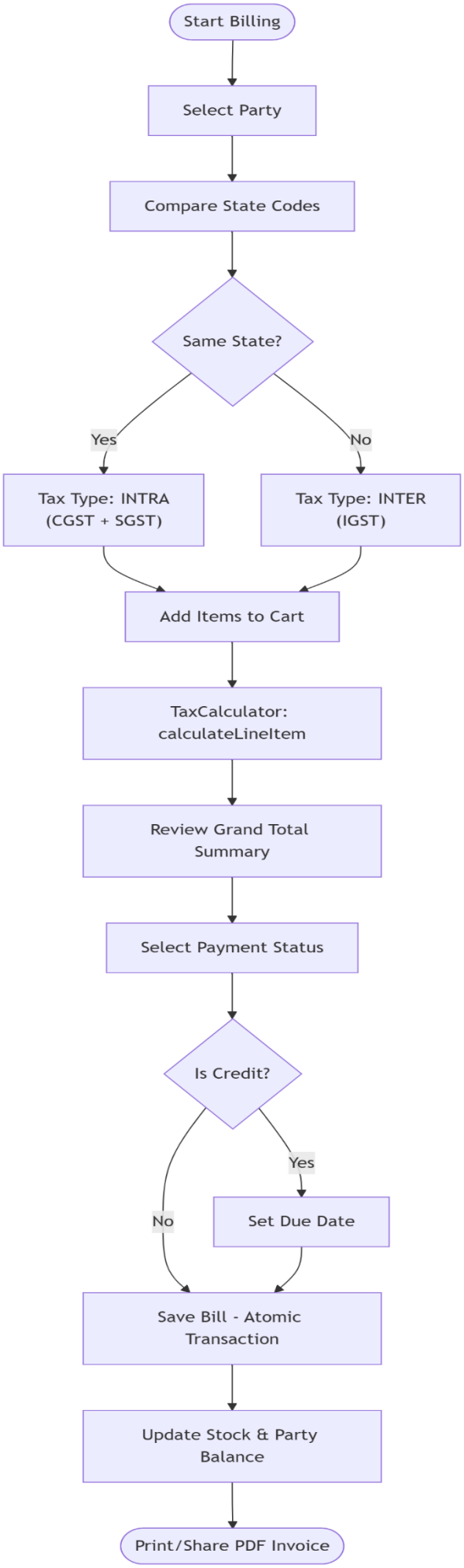
1. Activity Diagram

An Activity Diagram represents the dynamic flow of control within a system — similar to a flowchart. It shows the sequence of activities performed by the user and the system in response to actions. Three key workflows are described below.

a. **Activity: User Login & Onboarding Flow**

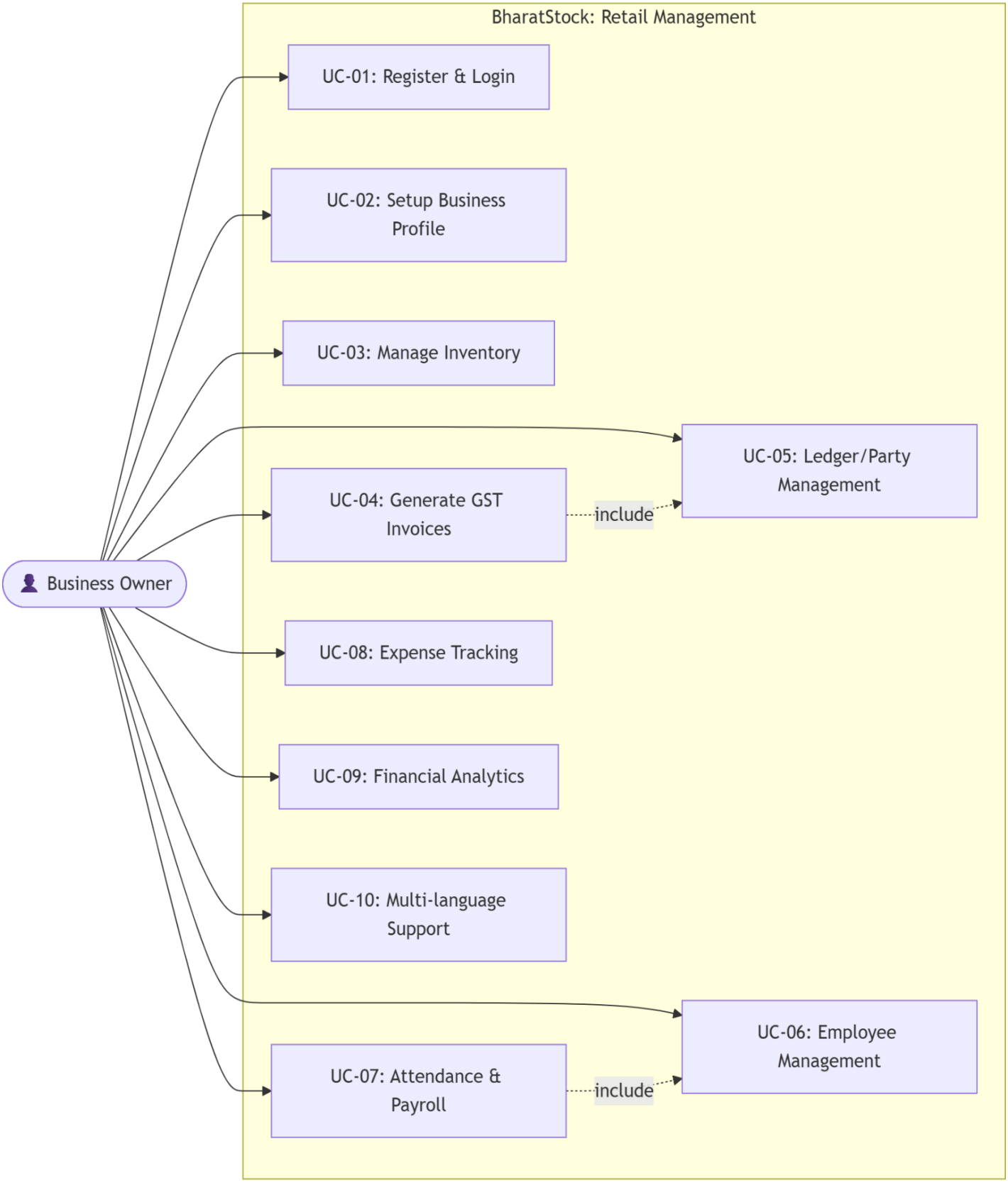


b. Activity: Billing Flow



2. Use Case Diagram

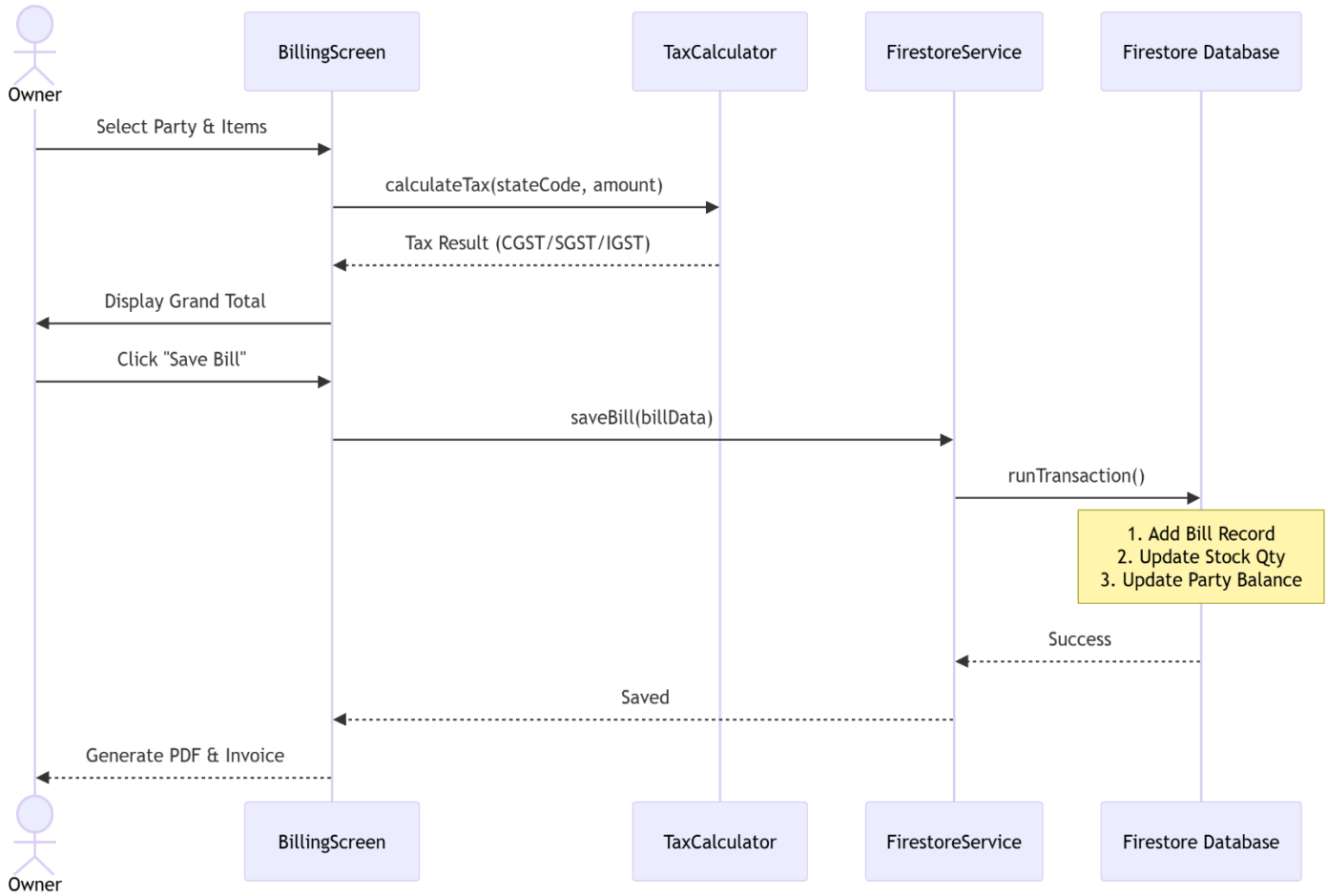
A Use Case Diagram shows the interactions between users (Actors) and the system. BharatStock is a single-actor system in its current build. The only actor is the Business Owner (Admin). The role field exists in the data model as a reserved field for a future multi-user feature, but role-based access control is not active in the current version.



3. Sequence Diagram

A Sequence Diagram shows the time-ordered exchange of messages between components when a specific use case is executed.

GST Bill Generation and Save



6.2 Database Design

BharatStock uses Google Cloud Firestore as its backend database — a document-based NoSQL database organized as a hierarchy of collections and documents. All business data follows a strict multi-tenant hierarchy rooted at /businesses/{businessId}/ to enforce data isolation between different registered shops.

FIRESTORE HIERARCHY

```
/users/{uid}                          ← Top-level user profiles


/businesses/{businessId}/
    ├── stocks/{stockId}
    ├── bills/{billId}
    ├── parties/{partyId}/
    │       └── transactions/{txnId}
```

```
├── staff/{staffId}/
│       ├── attendance/{YYYY-MM-DD}
│       └── salary/{MM_YYYY}
└── expenses/{expenseId}
```

6.2.1 Data Dictionary

A Data Dictionary formally defines every field stored in the database, its data type, constraint, and purpose.

Collection: users

Field Name	Data Type	Constraint	Description
uid	String	Primary Key (Firebase Auth UID)	Unique user identifier from Firebase Authentication.
email	String	Not Null, Unique	Email used for login.
fullName	String	Not Null	Display name of the business owner.
role	String	Enum: admin (default)	Reserved for future multi-user; currently always "admin".
businessId	String	FK → businesses	Links user document to their business namespace.
legalBusinessName	String	Not Null	Registered trade name as shown on invoices.
gstin	String	Optional, 15-char	15-character GST Identification Number.
businessStateCode	String	Not Null, 2-char	2-digit state code (e.g., 24 = Gujarat) for GST jurisdiction.
pan	String	Optional, 10-char	Permanent Account Number.
bankAccountNo	String	Optional	Bank account for invoice payment references.
bankIfsc	String	Optional	Bank IFSC code.
userImageUrl	String	Optional	Base64-encoded business logo for invoices.
signatureImageUrl	String	Optional	Base64-encoded digital signature stamp for PDFs.
onboardingDone	Boolean	Not Null	If true, skips Language and Onboarding screens on future launches.
createdAt	Timestamp	Not Null	Account creation timestamp (server-side).

Collection: businesses/{businessId}/stocks

Field Name	Data Type	Constraint	Description
id	String	Primary Key	Firestore auto-generated document ID.
name	String	Not Null	Product display name on invoices and search.
category	String	Not Null	Product category (e.g., Electronics, Clothing).
quantity	Double	Not Null, ≥ 0	Current available stock units.
price	Double	Not Null, > 0	Selling price (MRP or tax-exclusive based on isTaxInclusive).
costPrice	Double	Optional	Purchase price from supplier for profit margin.
hsnCode	String	Optional	HSN/SAC code for GST-compliant invoicing.
gstRate	Double	Not Null	GST slab: 0, 5, 12, 18, or 28 percent.
isTaxInclusive	Boolean	Not Null	True = MRP already includes GST; back-calculation applied.
lowStockNotify	Double	Not Null	In-app alert triggered when stock falls below this quantity.
itcEligible	Boolean	Not Null	Whether Input Tax Credit can be claimed on this purchase.
isVerifiedGstr2b	Boolean	Not Null	Whether purchase is matched against GSTR-2B for ITC verification.

createdAt	Timestamp	Not Null	Record creation timestamp.
updatedAt	Timestamp	Not Null	Last modification timestamp.

Collection: businesses/{businessId}/parties

Field Name	Data Type	Constraint	Description
id	String	Primary Key	Firestore auto-generated document ID.
name	String	Not Null	Party name (customer or supplier).
type	String	Enum: customer / supplier	Identifies whether the party is a buyer or vendor.
mobile	String	Not Null	Contact mobile number.
balance	Double	Not Null	Running ledger balance. +ve = they owe you; -ve = you owe them.
registrationType	String	Enum: unregistered/regular/composition	GST registration status.
gstin	String	Conditional, 15-char	Required for regular/composition GST parties.
stateCode	String	Not Null, 2-char	2-digit state code for intra/inter GST determination.
billingCity	String	Optional	City for billing address.
billingPincode	String	Optional	PIN code for billing address.
isShippingSame	Boolean	Not Null	If true, shipping address is same as billing.
bankAccount	String	Optional	Bank account for supplier payment tracking.
ifscCode	String	Optional	IFSC code for supplier bank.
isRegisteredOnApp	Boolean	Not Null	Whether party also uses BharatStock (for future B2B link).
createdAt	Timestamp	Not Null	Record creation timestamp.

Sub-Collection: parties/{partyId}/transactions

Field Name	Data Type	Constraint	Description
txnId	String	Primary Key	Auto-generated transaction document ID.
amount	Double	Not Null, > 0	Amount of the payment transaction.
type	String	Enum: payment/bill/adjustment	Nature of the transaction affecting the balance.
note	String	Optional	Reason or reference note for the transaction.
date	Timestamp	Not Null	Date on which the transaction occurred.

Collection: businesses/{businessId}/bills

Field Name	Data Type	Constraint	Description
id	String	Primary Key	Firestore auto-generated document ID.
billNumber	String	Not Null, Unique	Human-readable invoice number (e.g., INV-1773767131687).
partyId	String	FK → parties	Reference to the linked customer or supplier.
partyName	String	Not Null (snapshot)	Denormalized party name captured at time of billing.
date	Timestamp	Not Null	Invoice date selected by the user.
items	Array<Map>	Not Null	Embedded BillItem snapshots (name, qty, price, HSN, taxes, total).
totalAmount	Double	Not Null	Grand total inclusive of all GST components.

totalTaxableValue	Double	Not Null	Sum of taxable values before GST.
totalCgst	Double	Not Null	Total Central GST (intra-state bills only).
totalSgst	Double	Not Null	Total State GST (intra-state bills only).
totalIgst	Double	Not Null	Total Integrated GST (inter-state bills only).
billType	String	Enum: B2B / B2C	B2B = registered buyer; B2C = individual consumer.
customerGstin	String	Conditional	Buyer GSTIN — required for B2B invoices.
placeOfSupply	String	Not Null	Delivery state for GST jurisdiction determination.
taxType	String	Enum: intra / inter	"intra" if same state; "inter" if different state.
hsnSummary	Map<S,D>	Not Null	HSN-code-wise taxable values for GSTR-1 compliance.
reverseCharge	Boolean	Not Null	Whether Reverse Charge Mechanism applies.
paymentStatus	String	Enum: paid/credit/cash/online	Current payment status of the invoice.
dueDate	Timestamp	Conditional	Credit due date; only when paymentStatus = "credit".

Collection: businesses/{businessId}/staff

Field Name	Data Type	Constraint	Description
id	String	Primary Key	Firestore auto-generated document ID.
name	String	Not Null	Full name of the employee.
mobileNo	String	Not Null	Employee mobile number.
accountNo	String	Optional	Bank account for salary payment records.
salaryPerMonth	Double	Not Null, > 0	Agreed monthly salary in Indian Rupees.
joinDate	Timestamp	Not Null	Date the employee joined the business.

Sub-Collection: staff/{staffId}/attendance

Date-as-ID pattern (YYYY-MM-DD): Prevents duplicate entries and enables O(1) lookups for any date.

Field Name	Data Type	Constraint	Description
date (Doc ID)	String	Primary Key (YYYY-MM-DD)	Date of attendance. Used as document ID to prevent duplicates.
status	String	Enum: Present/Absent/Half-Day	Half-Day contributes 0.5 to the monthly salary calculation.

Sub-Collection: staff/{staffId}/salary

Month-as-ID pattern (MM_YYYY): One salary document per calendar month per staff member.

Field Name	Data Type	Constraint	Description
monthYear (Doc ID)	String	Primary Key (MM_YYYY)	Month-Year as doc ID ensures one record per month.
presentDays	Double	Not Null	Total present count for the month. Half-Day = 0.5.
totalWorkingDays	Integer	Not Null	Total working days in the month (days with attendance marked).
calculatedSalary	Double	Not Null	Formula: (presentDays / totalWorkingDays) × salaryPerMonth.
bonus	Double	Not Null, default: 0	Additional bonus on top of calculated salary.

status	String	Enum: Paid / Unpaid	Payment status for this month.
paymentDate	Timestamp	Conditional	Date of payment; populated only when status = "Paid".

Collection: businesses/{businessId}/expenses

Field Name	Data Type	Constraint	Description
id	String	Primary Key	Firestore auto-generated document ID.
title	String	Not Null	Short label for the expense (e.g., "Shop Rent", "Electricity").
amount	Double	Not Null, > 0	Expense amount in Indian Rupees.
date	Timestamp	Not Null	Date the expense was incurred.
category	String	Not Null	Category: Rent, Utilities, Salaries, Misc, etc.
type	String	Enum: recurring/one-time	Helps calculate monthly Profit & Loss accurately.

6.2.2 Database Relationship Diagram

The following table describes all parent-child relationships between Firestore collections in BharatStock.

Parent Collection	Relationship	Child Collection	Cardinality	Key Constraint
users	belongs to	businesses (via businessId)	Many : One	Every user must have a businessId after completing profile setup.
businesses	has many	stocks (sub-collection)	One : Many	Each stock item is scoped to exactly one business.
businesses	has many	parties (sub-collection)	One : Many	Each party is scoped to one business; balance is per-business.
parties	has many	transactions (sub-collection)	One : Many	Each payment transaction belongs to one party.
businesses	has many	bills (sub-collection)	One : Many	Bills are scoped to the generating business.
bills	references	parties (via partyId)	Many : One	Soft FK; party name/mobile are also snapshotted for immutability.
bills.items[]	references	stocks (via stockId in map)	Many : One	Soft FK; stock data is snapshotted at billing time. Immutable.
businesses	has many	staff (sub-collection)	One : Many	Each staff member belongs to exactly one business.
staff	has many	attendance (sub-collection)	One : Many	One attendance document per calendar day (doc ID = YYYY-MM-DD).
staff	has many	salary (sub-collection)	One : Many	One salary record per month per employee (doc ID = MM_YYYY).
businesses	has many	expenses (sub-collection)	One : Many	All operational expenses scoped to one business.

Multi-tenant Design: BharatStock uses a sub-collection hierarchy rooted at /businesses/{businessId}/. This enforces Firestore Security Rules at the path level, guaranteeing that Business A can never read or write Business B's data.

6.3 E-R Diagram

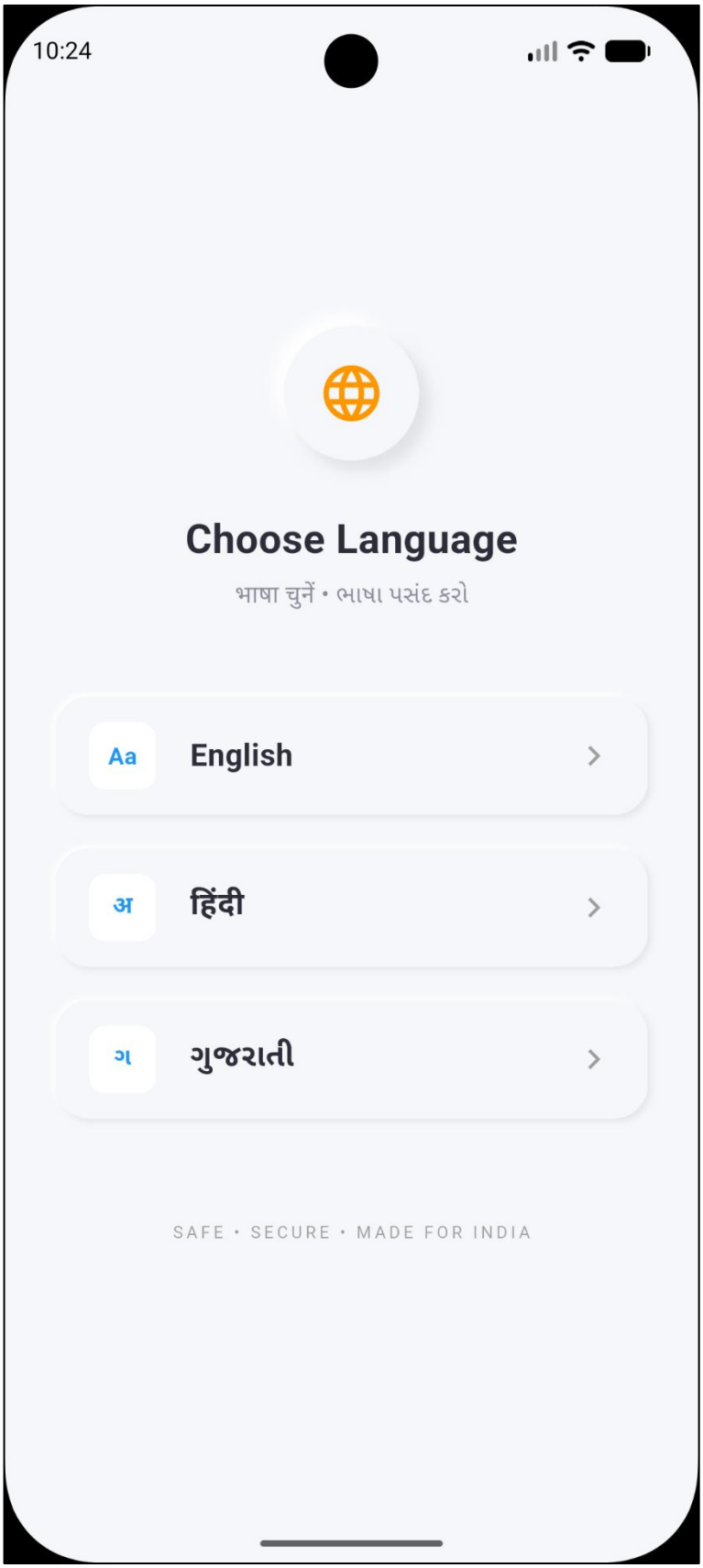
6.4 User Interface Design

BharatStock employs a Claymorphic Design Language — a modern visual style characterized by soft shadows, rounded corners, slightly raised card surfaces, and warm-toned background fills. This design makes the app feel tactile and approachable for non-technical business owners.

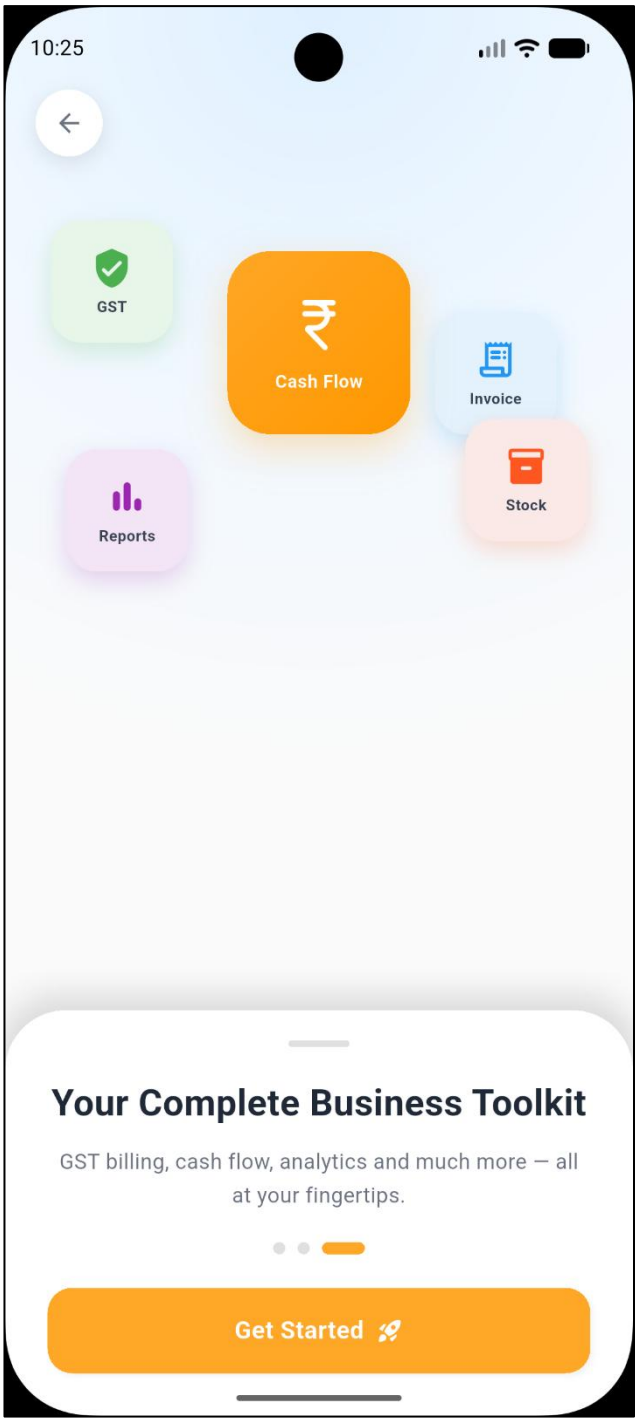
Screens

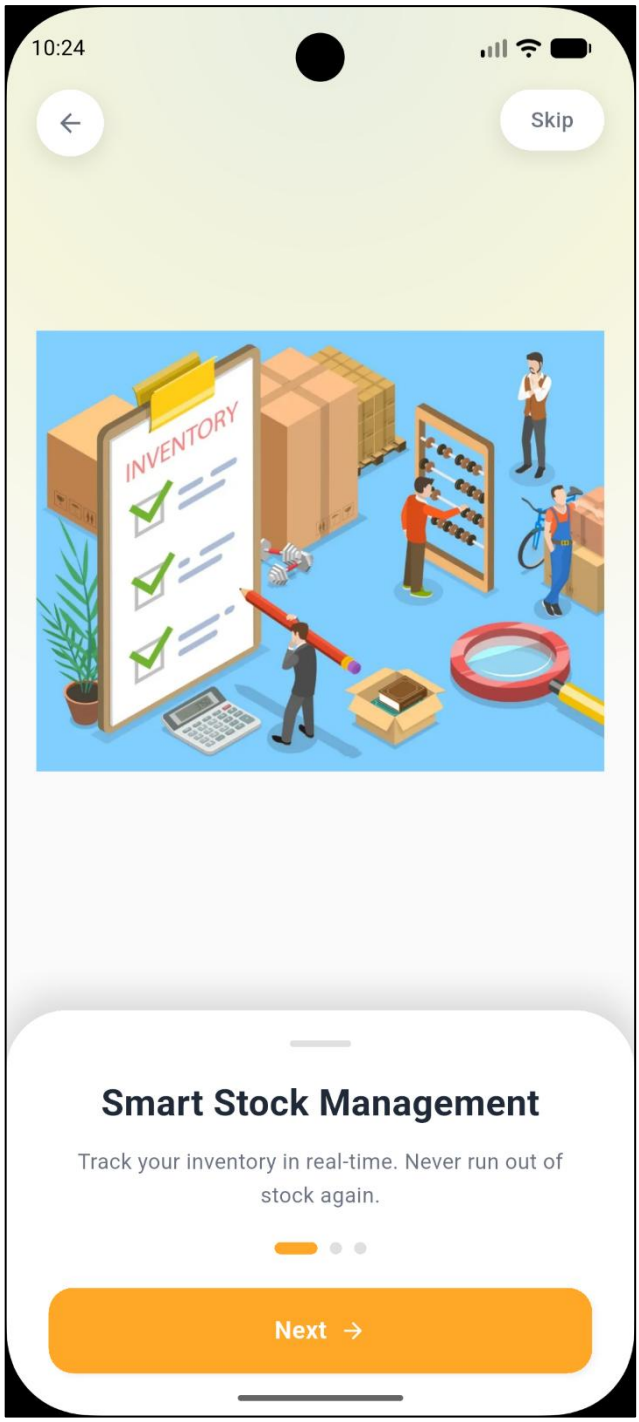
1. Splash Screen

2. Language Selection Screen

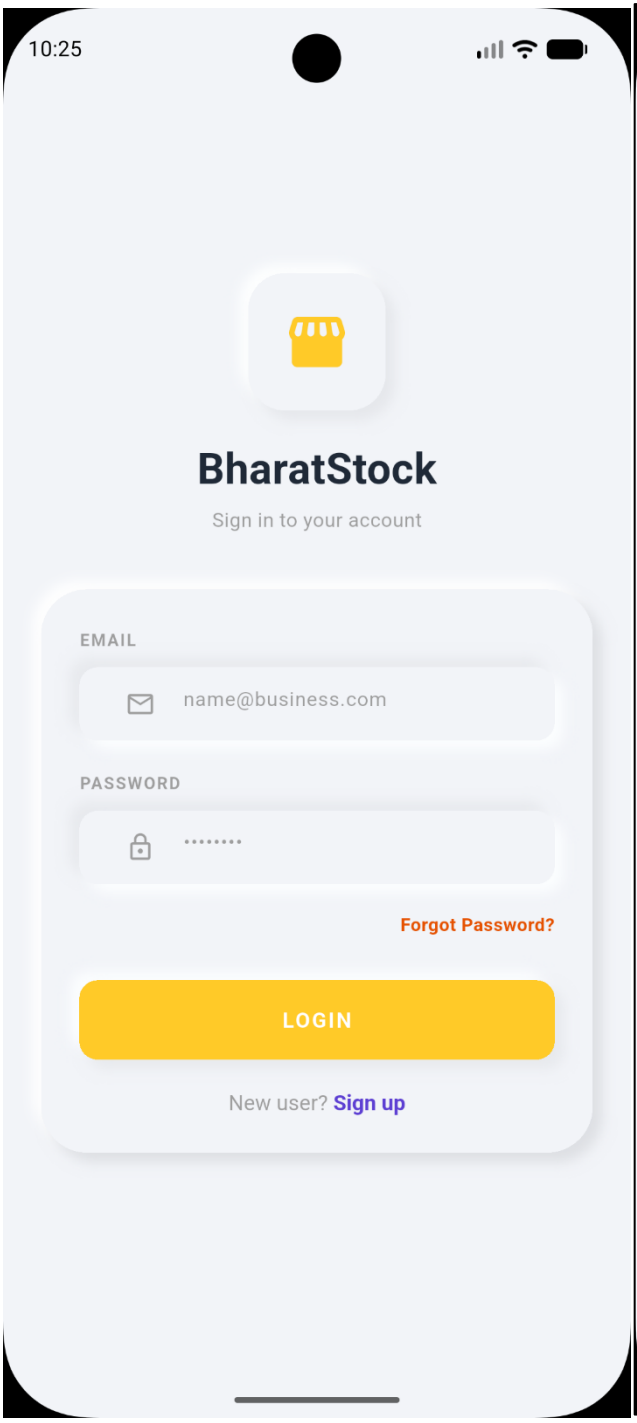


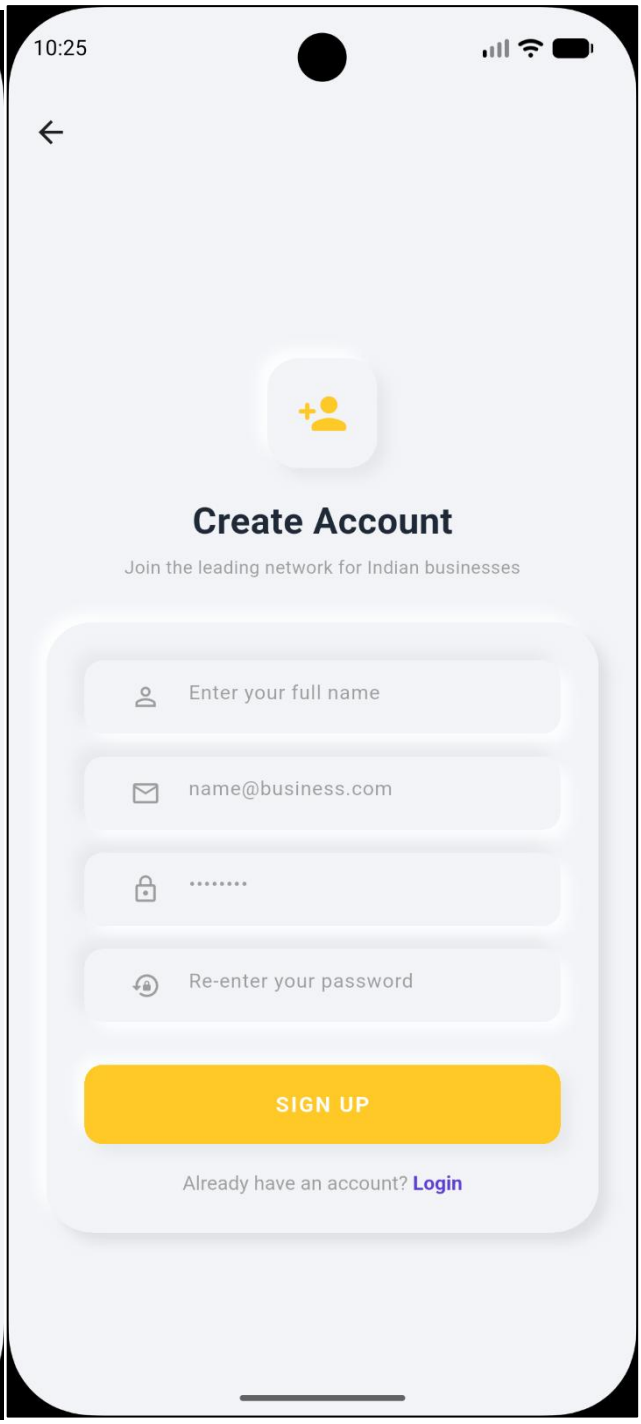
3. Onboarding Screen



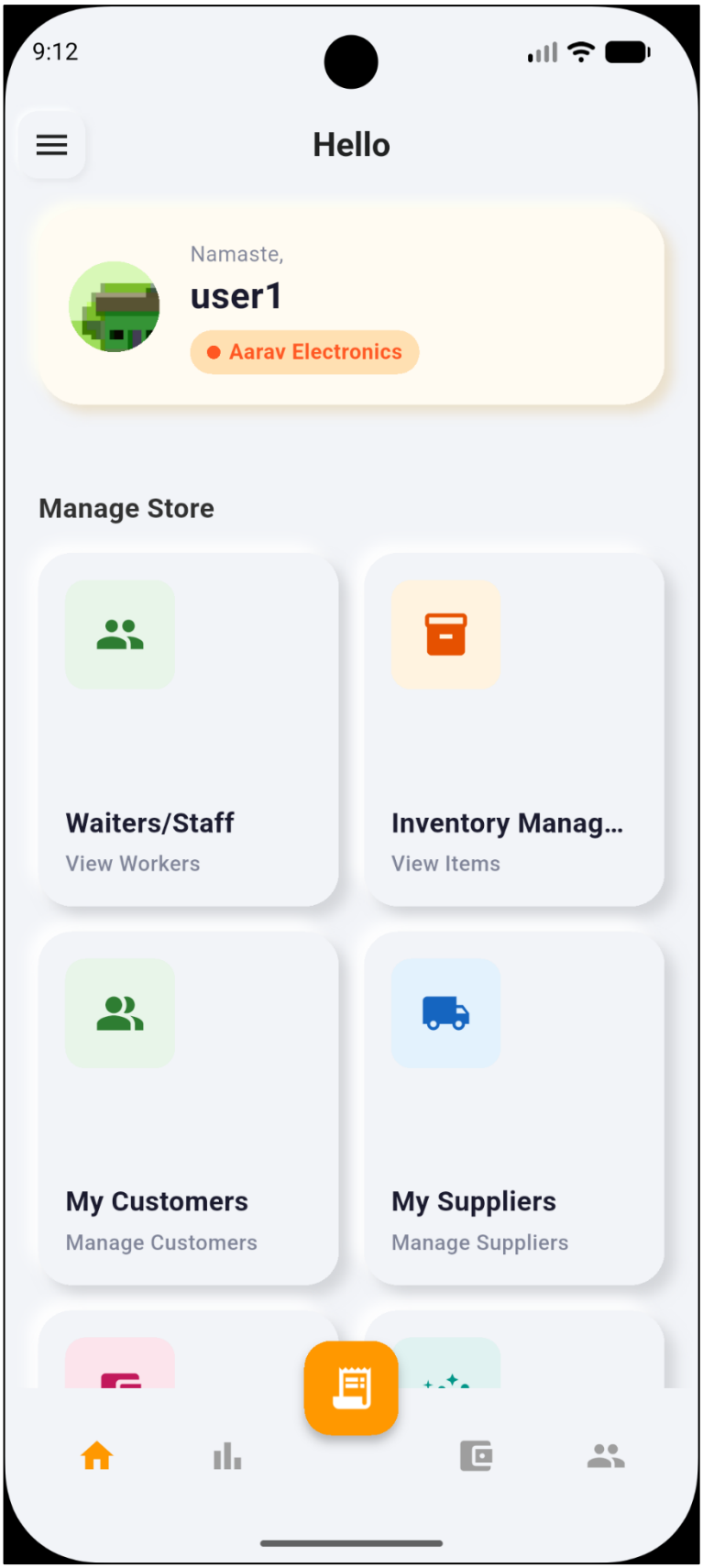


4. Login / Sign Up

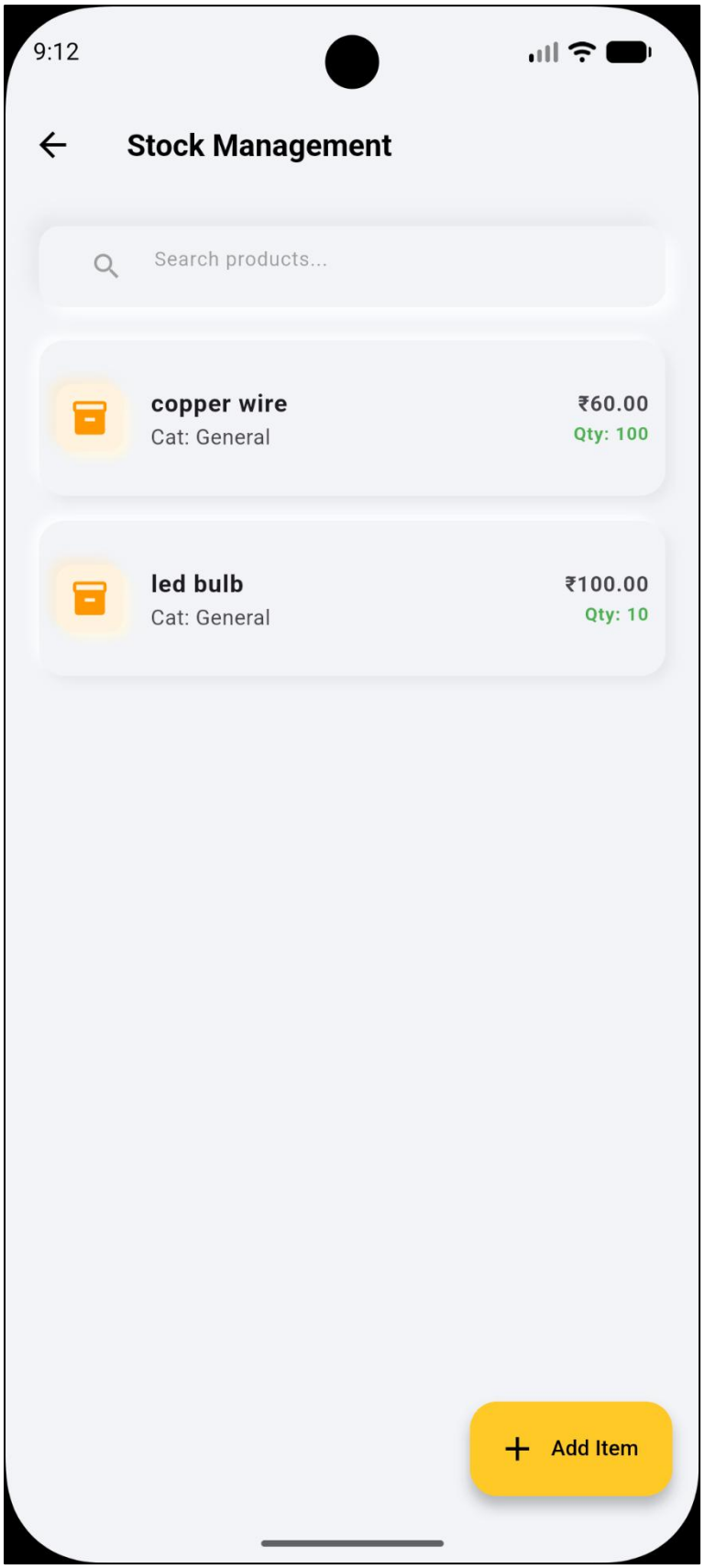




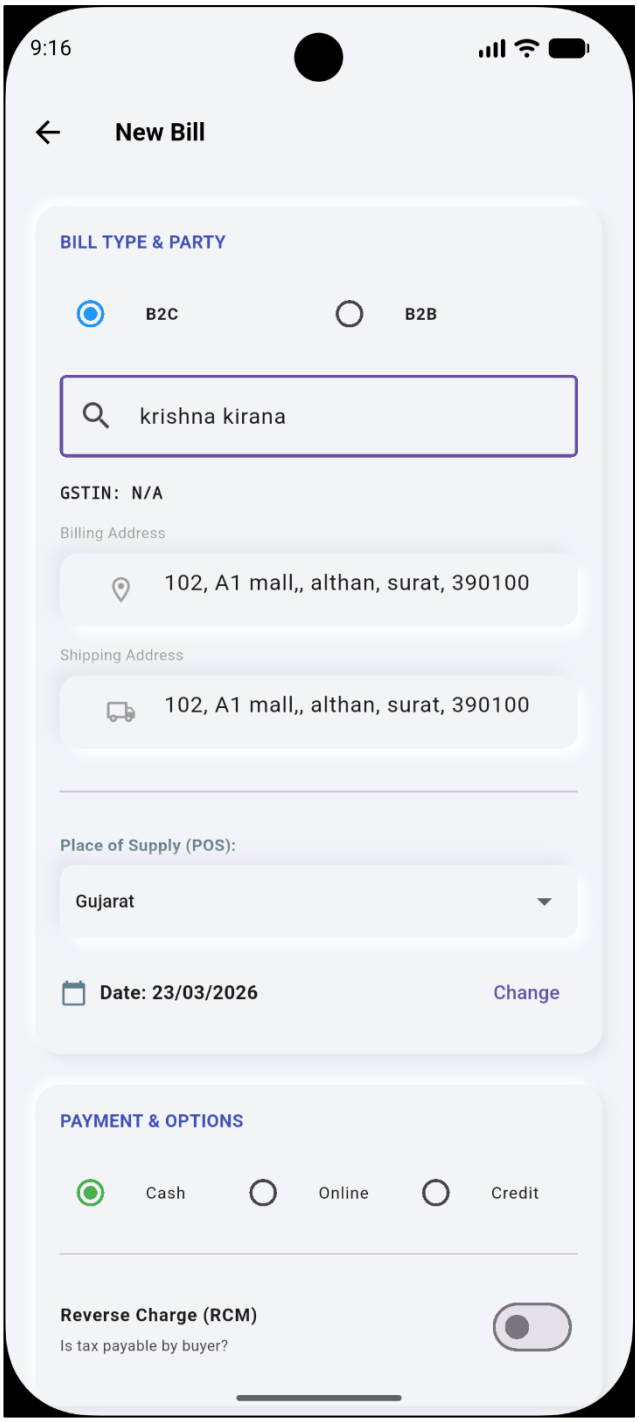
5. Home Dashboard

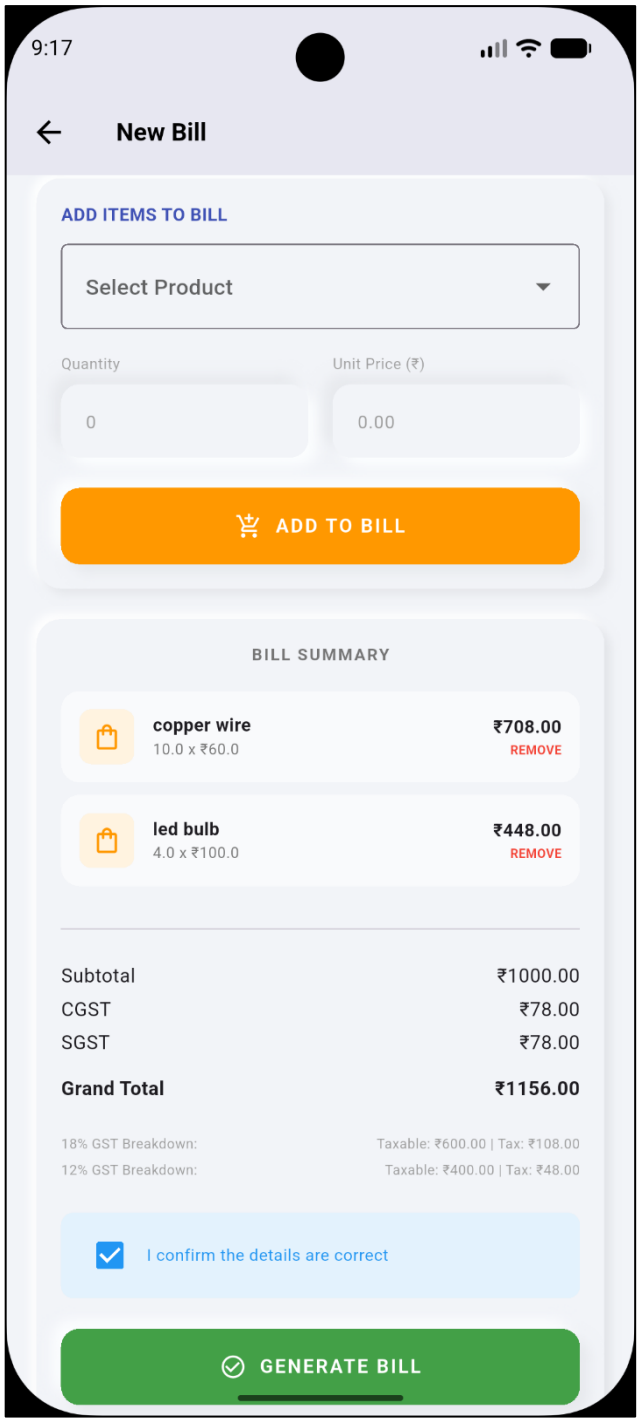


6. Inventory Screen

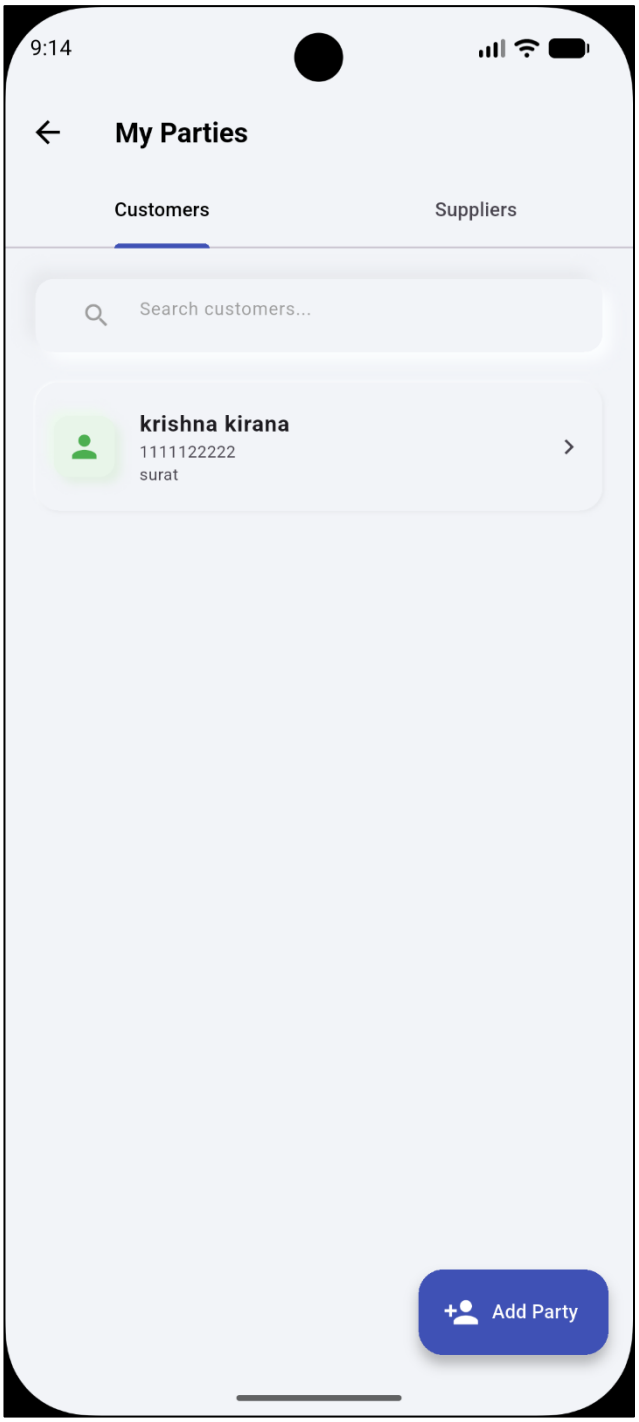


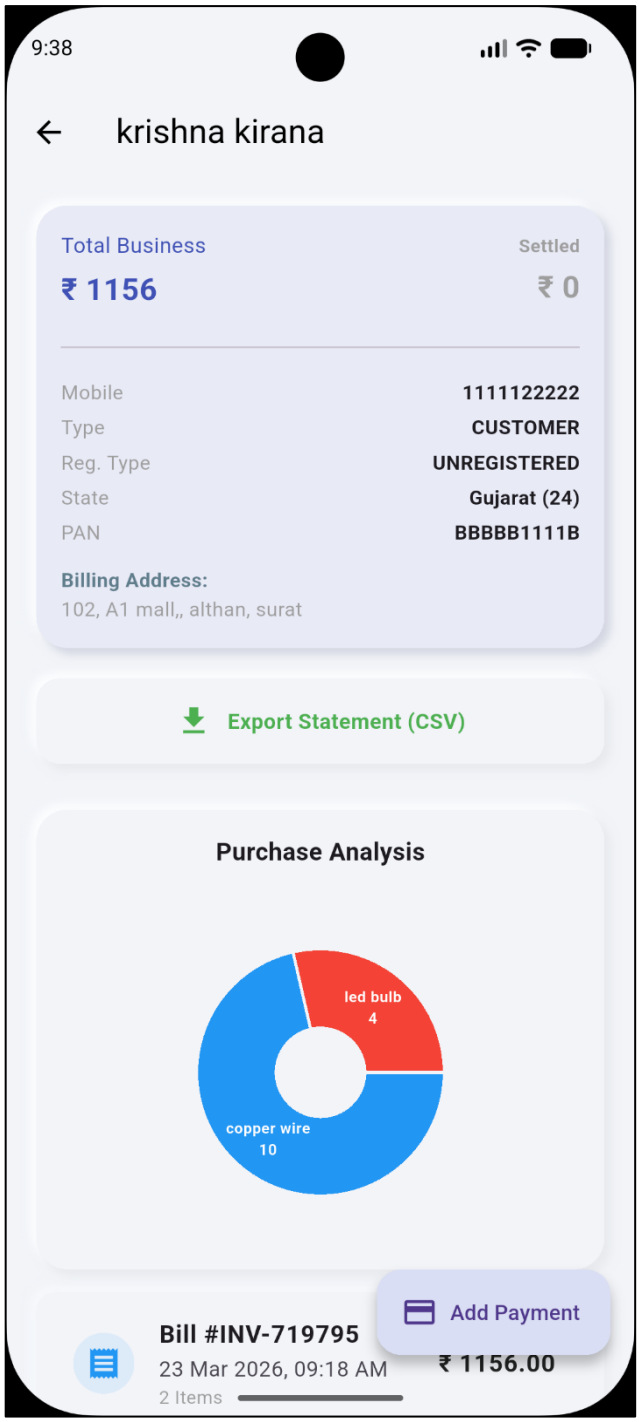
7. Bill Generation Screen



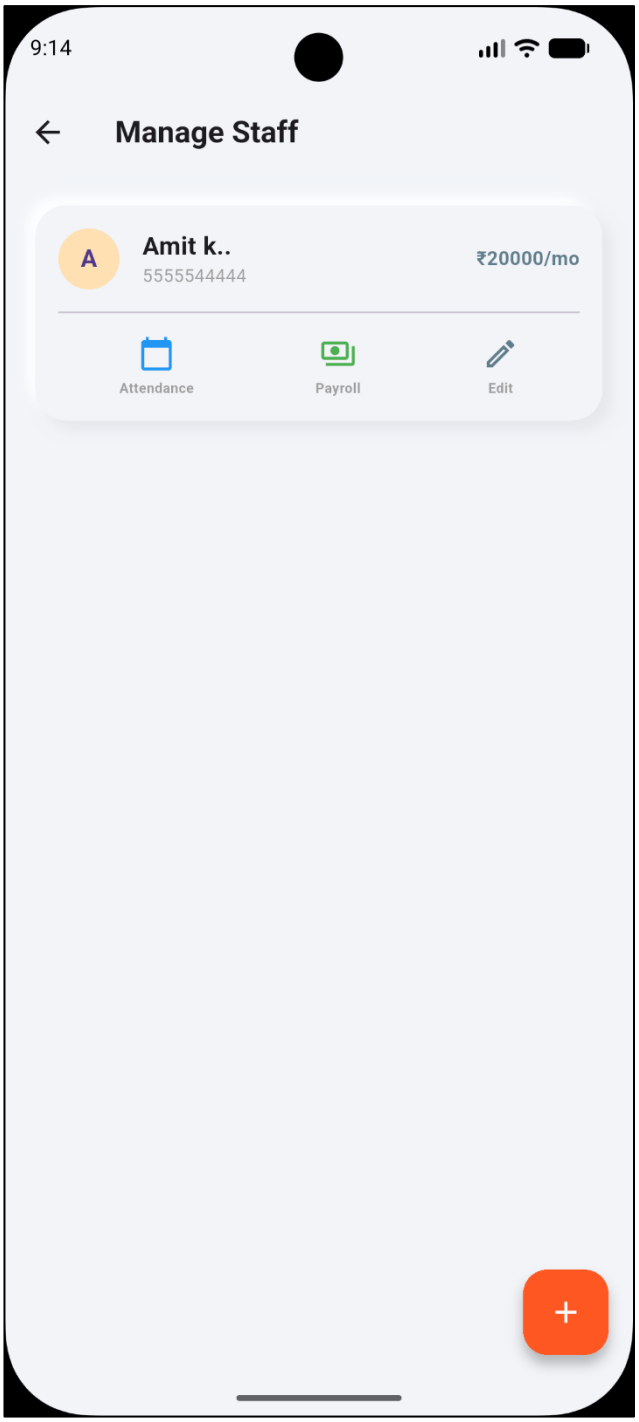


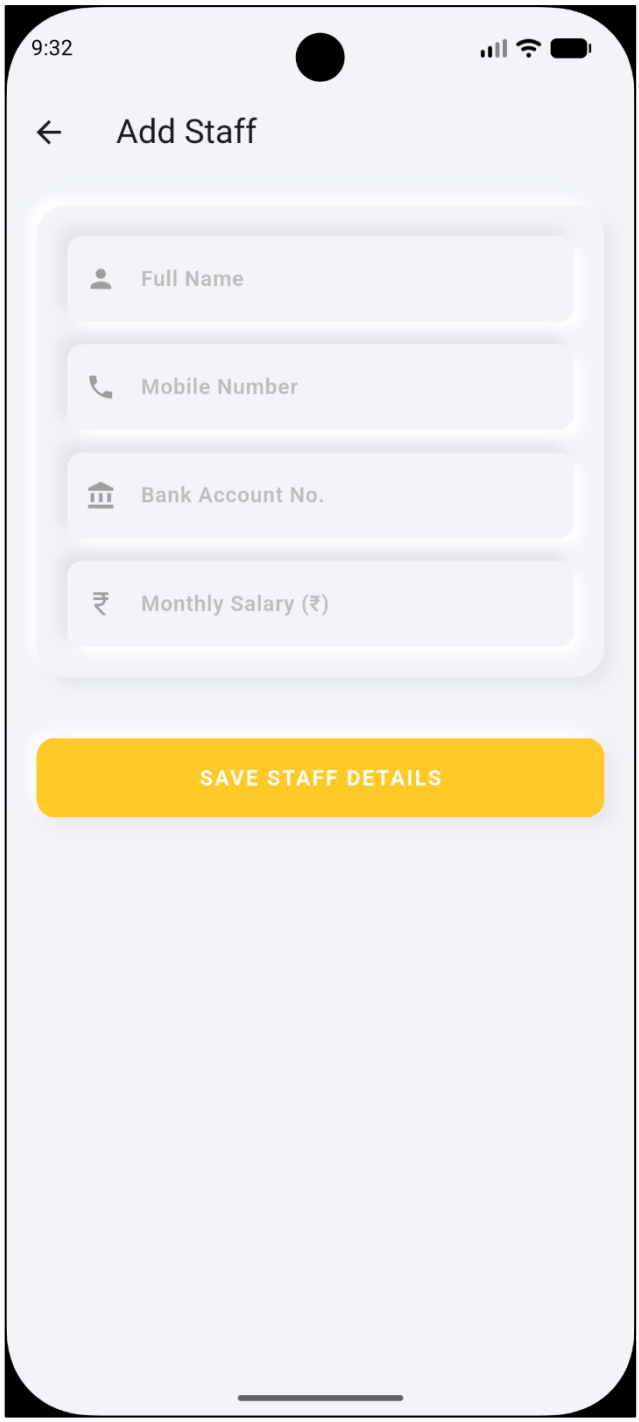
8. Party Management Screen



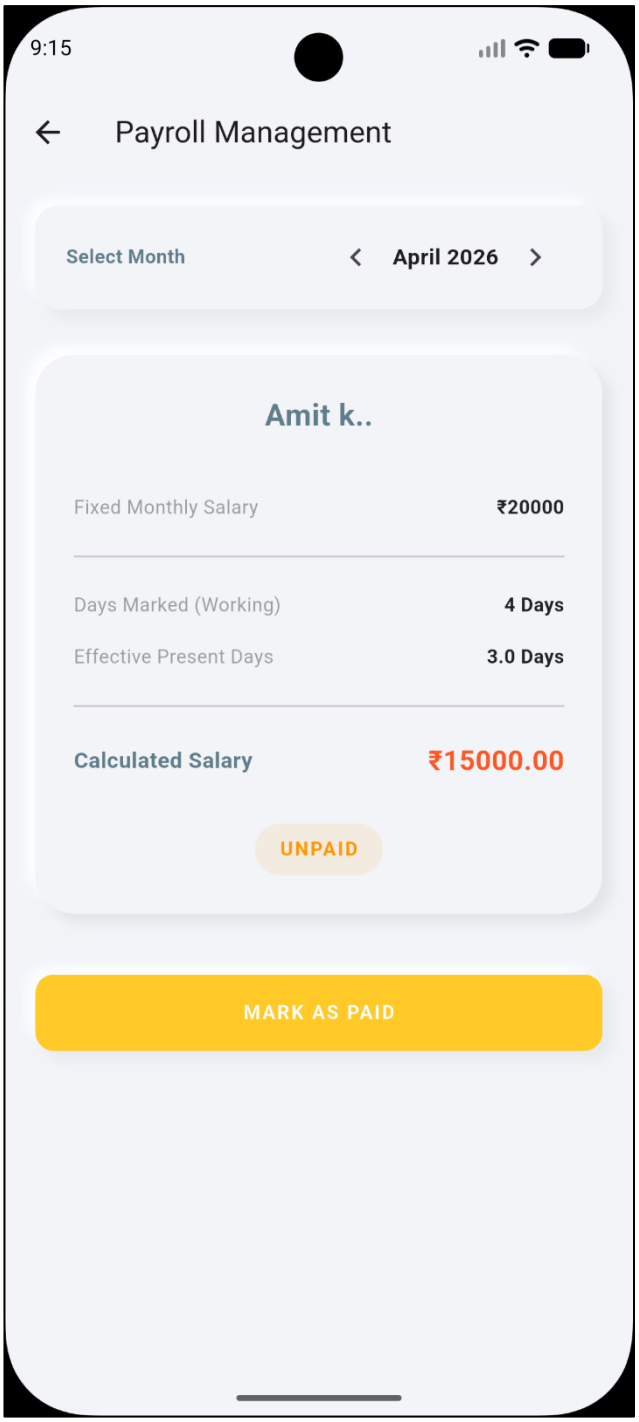


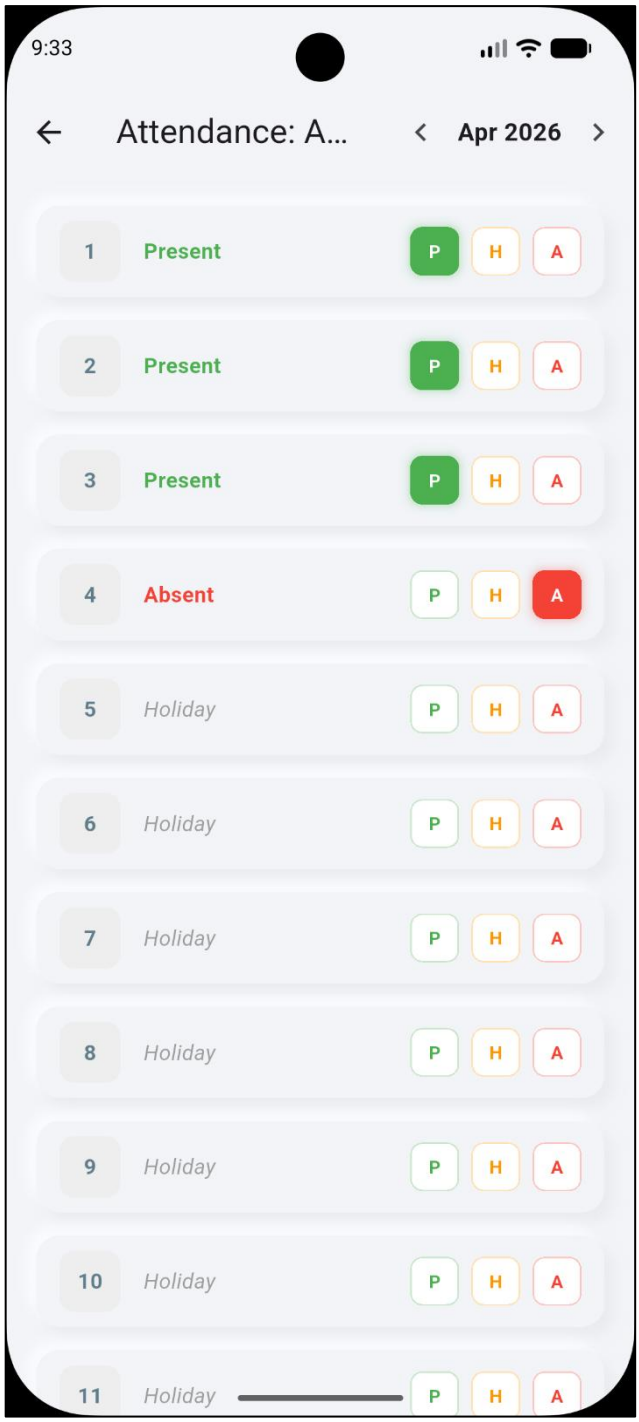
9. Staff Management Screen



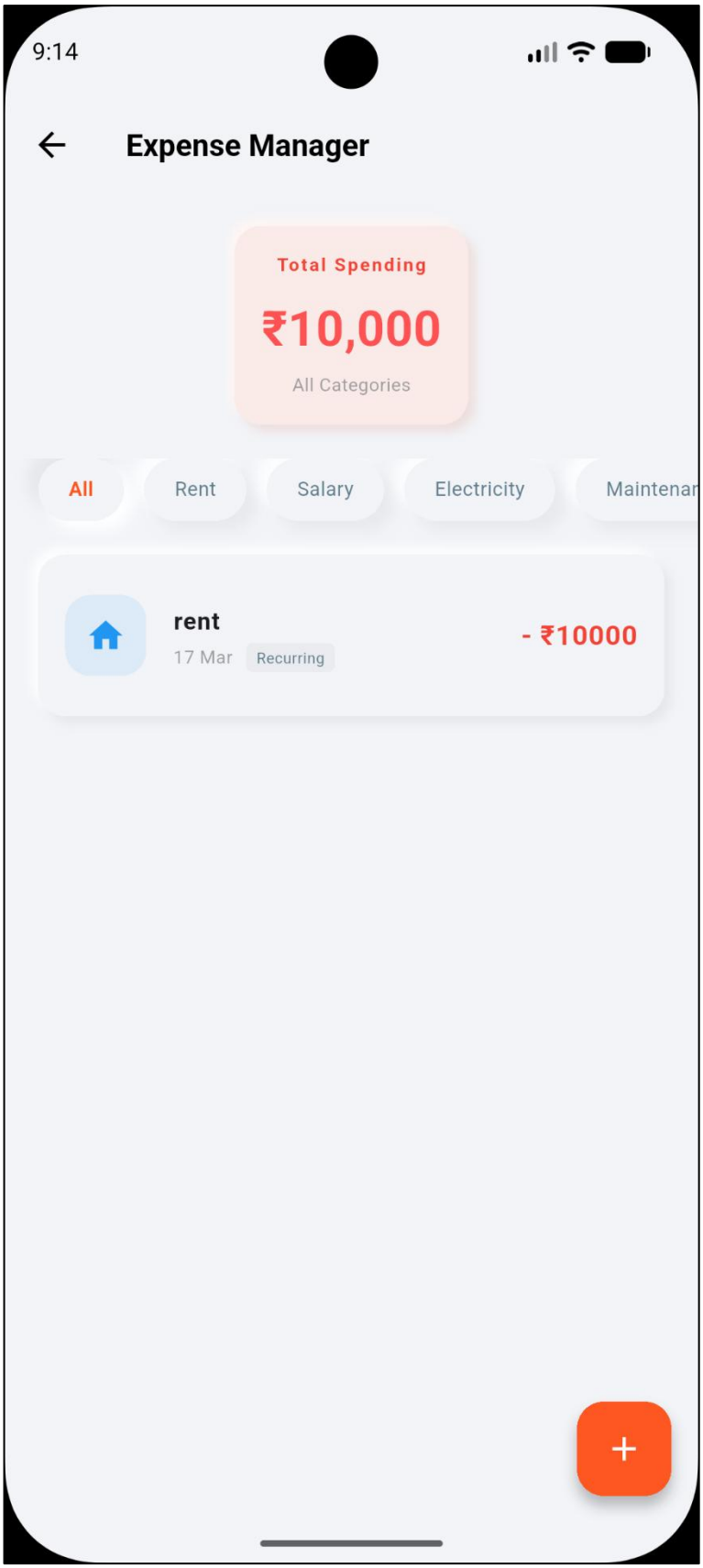


10. Attendance & Payroll Screen

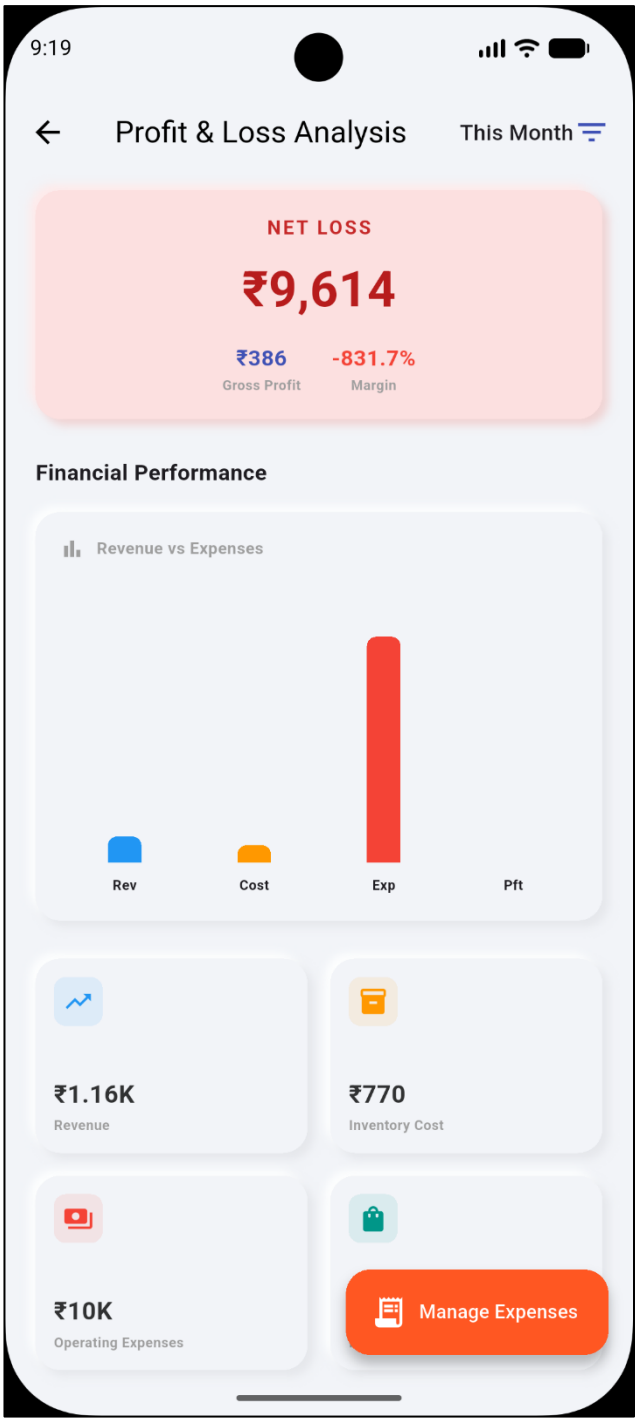


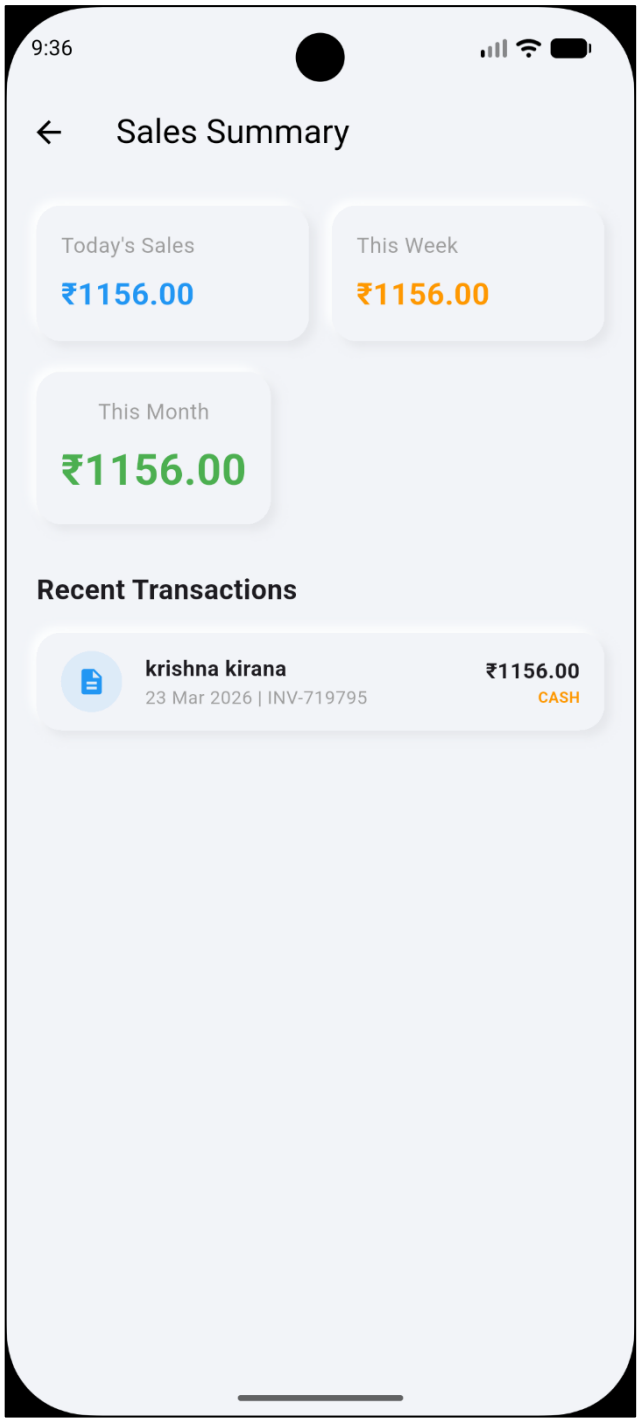


11. Expenses Screen



12. Analytics Screens





7. System Testing

System testing is the systematic process of evaluating a complete, integrated system to verify that it meets its specified requirements and performs correctly under all anticipated operating conditions. For BharatStock, a three-tier testing strategy was employed: Unit Testing for individual functions and components, Integration Testing for module-to-module interactions, and System Testing for end-to-end functional validation.

7.2 Integration Testing

Integration testing validates that the BharatStock application's independently developed modules interact correctly when combined. The most critical integration points in BharatStock are the interfaces between the Flutter UI layer, the Firestore database, and the Firebase Cloud Functions.

1. **Purpose of Integration Testing**

   Integration tests check if data flows correctly between different parts of the app. It ensures that Firebase, UI components, and user interactions work seamlessly. The goal is to identify bugs related to module dependencies and communication. This testing ensures a smooth user experience across all features.

2. **Test Cases**

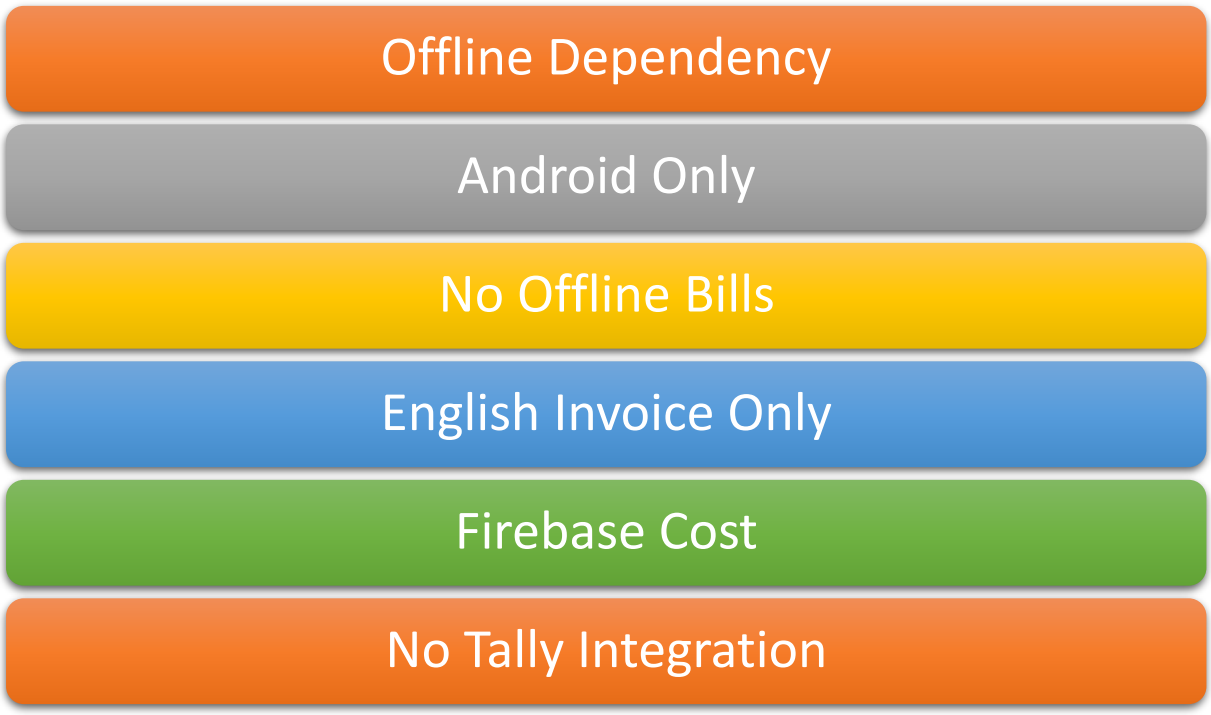
Test Case ID	Integration Point	Test Scenario	Expected Result	Result
IT.1.1	Auth → Role Routing	Logged in	Redirected to Dashboard with full menu visible	PASS
IT.2.1	Billing → Stock	Generate bill for 5 units of Product A (stock: 20)	Product A stock updated to 15 in Firestore within 2 seconds	PASS
IT.3.1	Invoice → PDF → Storage	Generate invoice and trigger PDF creation	PDF file created in Firebase Storage, URL stored in bill document, file accessible via URL	PASS
IT.4.1	Billing → Party Balance	Generate a credit bill for ₹3,000 against a customer party	Party balance updated to +₹3,000 atomically in Firestore; transaction record created	PASS
IT.5.1	Low-Stock Alert → FCM	Bill reduces Product B stock below minimum threshold	Cloud Function triggers within 10s, FCM notification delivered to Owner device	PASS
IT.6.1	Attendance → Salary Calculation	Mark staff as Present for 22 days; trigger salary view	Salary screen shows calculated salary = (22/26) × monthly salary	PASS

3. **Benefits of Integration Testing**

   Integration testing reduces system failures, improves performance, and enhances stability. It helps developers identify data flow issues between modules before deployment. Ensuring smooth interactions between components leads to a better user experience and reliability.

8. Limitations

While BharatStock represents a significant advancement over existing paper-based and basic digital retail management tools, it is important to transparently document the current limitations of the system. These limitations are acknowledged by the development team and are targeted for resolution in subsequent development phases:

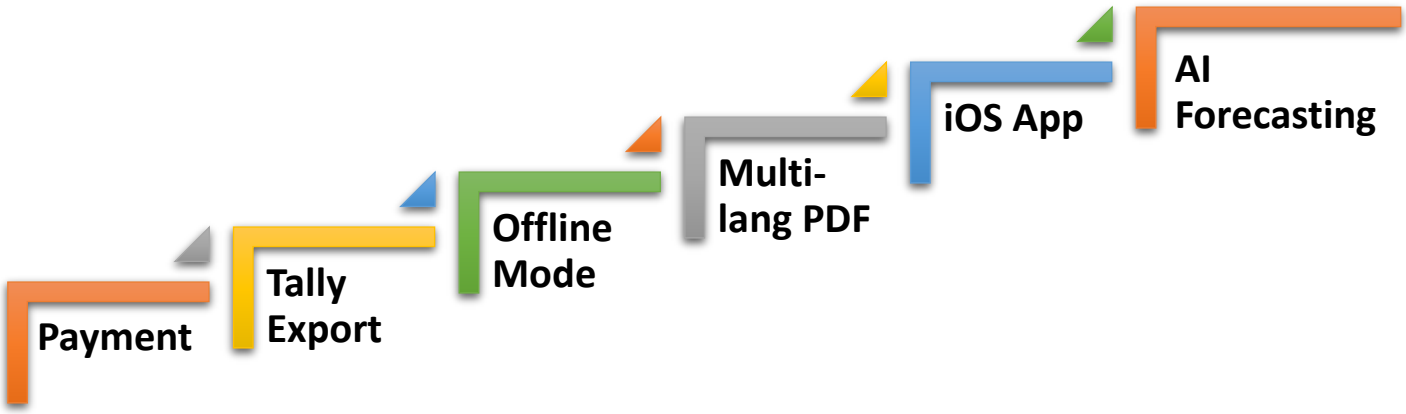


1. **Internet Connectivity Dependency:** As a cloud-first application built on Firebase's real-time infrastructure, BharatStock requires a minimum 2G mobile data connection to perform write operations such as saving bills, updating stock, and dispatching B2B interactions. While read operations benefit from Firestore's built-in offline persistence cache — allowing users to browse existing products and view past invoices — new transactions cannot be finalized and synced without network access. This is a meaningful limitation in regions of India where mobile connectivity remains unreliable, particularly during monsoon seasons when network infrastructure may be disrupted.
2. **Android-First Platform Support:** The current release version of BharatStock is optimized and tested for Android devices. While Flutter's cross-platform architecture theoretically enables iOS compilation from the same codebase, the iOS build requires additional configuration (Apple Developer account, code signing certificates, App Store review compliance) and device-specific testing that extends beyond the scope of the current project phase. iOS support is planned as an explicit deliverable in the next development cycle.
3. **No Offline New-Bill Creation:** While product browsing and historical bill viewing are available offline, creating and saving new bills is currently blocked when the device has no internet connectivity. Implementing a robust offline bill queue — where bills are created locally and atomically synced and reconciled when connectivity is restored — requires careful handling of stock quantity race conditions and invoice ID sequencing, and is planned as a future enhancement.

4. **Single-Language Invoice:** Currently, PDF invoices are generated in English only. The application UI supports Hindi and Gujarati through Flutter's localization system, but the PDF generation library requires additional configuration to render Devanagari and Gujarati Unicode characters correctly in the PDF output. Multi-language invoice generation is a planned enhancement.
5. **Firebase Pricing Threshold:** While Firebase's free Spark tier is sufficient for early-stage deployment, businesses with high transaction volumes (above approximately 20,000 Firestore reads/writes per day) will encounter Firebase's paid Blaze plan thresholds. For medium-to-large retailers, operational costs on Firebase's pay-as-you-go Blaze plan must be factored into the total cost of ownership.
6. **No Built-In Accounting Integration:** BharatStock currently does not provide direct integration with popular Indian accounting software packages such as Tally Prime or QuickBooks. Financial data must be exported as CSV reports and manually imported into these systems by the user, which introduces an additional step in the accounting workflow for retailers who rely on separate accounting software.

9. Future Enhancement

The BharatStock development roadmap includes a rich set of planned enhancements that will significantly expand the system's capabilities, reach, and intelligence. These enhancements are prioritized based on the feedback collected during user testing sessions and the evolving regulatory environment:



1. **AI-Powered Inventory Intelligence:** Integrating a machine learning-based demand forecasting engine that analyses historical sales velocity, seasonal trends, and external market signals (such as upcoming festivals or holidays) to predict which products are likely to face stock shortages in the coming 7–14 days. This predictive capability will allow shop owners to proactively place procurement orders before stock-outs occur, eliminating lost revenue events.
2. **Integrated Digital Payments:** Incorporating the Razorpay or PhonePe SDK to enable direct digital payment collection through UPI, card, and net banking channels within the BharatStock bill confirmation flow. Upon payment completion, the system will automatically mark the invoice as 'paid' and send a payment confirmation receipt to the customer via WhatsApp — creating a fully paperless, end-to-end digital transaction experience.
3. **Tally Prime Direct Export:** Developing a certified XML export module that produces Tally-Prime-compatible voucher files from BharatStock's sales and purchase data. This integration will allow retailers who maintain a parallel Tally accounting system to automatically synchronize their BharatStock transaction data without manual re-entry, eliminating a significant source of data entry duplication and error.
4. **Multi-Language PDF Invoice Generation:** Extending the PDF invoice generation engine to render invoices in Hindi and Gujarati in addition to English, using proper Unicode font embedding and right-to-left text layout where applicable. This will allow retailers to provide professionally formatted invoices in their customers' preferred regional languages.
5. **Offline-First Architecture with Conflict Resolution:** Redesigning the bill creation and stock update workflows to operate fully offline, with intelligent conflict resolution logic that handles concurrent stock modifications from multiple staff members when connectivity is restored. This will make BharatStock viable for use in remote retail environments where internet access is intermittent.
6. **iOS App Release:** Building, configuring, and submitting the BharatStock iOS application to the Apple App Store, making the platform accessible to the segment of Indian business owners who use Apple devices.

7. **E-Invoice (IRN) Generation:** Integrating with the Indian Government's Invoice Registration Portal (IRP) API to support the mandatory e-Invoice generation requirement for businesses above the applicable turnover threshold, automatically generating Invoice Reference Numbers (IRNs) and QR codes for qualifying invoices.

10.References

10.1 Webliography

1. Flutter Official Documentation – https://docs.flutter.dev
2. Dart Programming Language Official Docs – https://dart.dev/guides
3. Google Firebase Documentation – https://firebase.google.com/docs
4. Cloud Firestore Documentation – https://firebase.google.com/docs/firestore
5. Flutter pub.dev – pdf package – https://pub.dev/packages/pdf
6. Flutter pub.dev – intl package (localization) – https://pub.dev/packages/intl
7. Flutter pub.dev – firebase_core, cloud_firestore, firebase_auth – https://pub.dev/publishers/firebase.google.com/packages